CC 102 Java Programming Module

Generated Jul 4, 2026 5:08 PM

Description

A foundational Java programming module covering Java concepts, development environment setup, program structure and syntax, algorithms, flowcharts, pseudocode, operators, user input, output, and basic file handling.

Learning Objectives

- Explain fundamental Java concepts and the role of Java tools.
- Set up and test a Java development environment.
- Read, label, and write basic Java program structures.
- Design algorithms, flowcharts, and pseudocode before coding.
- Use variables, data types, and operators to solve beginner programming problems.
- Read input, display clear output, and use basic file input/output in Java.

Lesson Overview

This module is a foundational CC 102 Java programming learning pack. Lessons move from Java concepts and setup, to program structure and syntax, algorithmic thinking, flowcharts, pseudocode, Java expressions, input/output, and basic file handling.

Detailed Discussion

The course uses gradual release: teacher modeling, example analysis, guided practice, independent application, and reflection. The sequence follows Acquisition, Making Meaning, and Transfer so students learn why planning and code structure matter.

Examples

Hello World, code labeling, syntax correction, everyday algorithms, flowcharts, pseudocode, Java variables, arithmetic expressions, Scanner input, formatted output, and text file reading/writing.

Student Activities

Quick maps, code labeling, spot-the-error tasks, algorithm writing, flowcharting, pseudocode revision, expression practice, input/output exercises, and a mini file-handling project.

Resources / References

Reference inspiration: TLE-ICT ReEcho Hub MATATAG-aligned workflow. Suggested tools: JDK, IntelliJ IDEA Community Edition or VS Code with Java extensions, Java documentation, printed flowchart guides, worksheets, and sample code snippets.

Java Fundamentals, Development Environment, And Algorithms

Overview

We learn Java concepts, prepare the development environment, read and write basic Java program structure, correct syntax errors, and design algorithms before coding.

Essential Question

How do programmers turn a problem into clear, testable instructions that Java can run?

Performance Task

Prepare and demonstrate a working Java environment, annotate a beginner Java program, correct its syntax errors, and design a tested algorithm for a simple problem.

Success Criteria

- Java concepts and development tools are explained accurately.
- The environment runs a simple Java program successfully.
- Program parts and syntax corrections are clearly identified.
- The algorithm contains complete, ordered, and testable steps.
- The learner explains evidence from testing and revision.

LESSON 1

Lesson 1: Java Concepts And Real-World Programming

What We'll Learn

By the end of this lesson, we can: - Define Java in beginner-friendly language. - Identify real-world systems where Java can be used. - Explain why Java programs need code, tools, testing, and revision.

Words We'll Use

- Java - a programming language used to create applications.
- Program - instructions a computer follows.
- Application - software that performs tasks for users.
- Developer - a person who designs, writes, tests, and improves programs.
- Output - result produced by a program.

Before We Start

What apps or systems do you use every day? Which of them might need programming? List school systems, mobile apps, games, ATMs, websites, and enrollment tools.

Let's Understand

Java toolchain guide Let's work through Lesson 1: Java Concepts And Real-World Programming together. Our focus is the purpose of Java, where Java is used, and why it is useful for beginners. We will move from meaning, to examples, to practice, then to a task we can explain. What this lesson means: - Java is a programming language we use to write instructions for a computer. - A Java program is written as source code, saved in a `.java` file, compiled, then run. - The JDK gives us the tools for building Java programs; the JVM runs the compiled bytecode. - An IDE helps us write, organize, run, and debug code in one workspace. - A beginner Java program usually has a class, a main method, statements, braces, and comments. - We read code from the outside container toward the inside instructions, then we trace what will display. How we study it step by step: - First, we connect the lesson to a familiar situation: What apps or systems do you use every day? Which of them might need programming? List school systems, mobile apps, games, ATMs, websites, and enrollment tools. - Next, we name the important parts so everyone uses the same vocabulary. - Then, we study What Java is and ask: what does it do, where do we see it, and how can we check it? - Then, we study Where Java is used and ask: what does it do, where do we see it, and how can we check it? - Then, we study Programmer mindset and ask: what does it do, where do we see it, and how can we check it? - Then, we study From problem to program and ask: what does it do, where do we see it, and how can we check it? - After that, we compare a correct example with a common wrong version. - We trace the example slowly before running, drawing, or submitting anything. - Finally, we explain the result in our own words so the activity is not just copying. Rules and patterns we should remember: - Rule: Java is case-sensitive, so `System` and `system` are not the same. Example: `System.out.println("Hi");` works, but `system.out.println("Hi");` causes an error. Check: Which word must start with a capital S? - Rule: Text output must be inside double quotation marks. Example: `System.out.println("Hello, Java!");` displays Hello, Java! Check: What text will appear on the screen? - Rule: Many Java statements end with a semicolon. Example: `int score = 90;` is complete, but `int score = 90` is missing its ending mark. Check: What symbol should we add at the end? - Rule: Braces { and } must be paired because they show where a block starts and ends. Example: `public class Demo { public static void main(String[] args) { } }` has matching class and main braces. Check: How many opening and closing braces can we count? - Rule: Comments explain code for humans and are ignored when the program runs. Example: `// This line prints a greeting` helps us understand the next statement. Check: Will a comment display in the console? Common mistakes we will watch for: - Forgetting capitalization in Java keywords or class names. - Missing quotation marks, semicolons, or closing braces. - Thinking comments run as code. - Changing code without predicting the output first. - Submitting a screenshot without explaining what happened. Mini-check before practice: - What part of the Java program is the container? - Where does the program start running? - What line displays output? - What syntax rule should we check before running? When we move to practice, our goal is not only to get an answer. Our goal is to explain the thinking: what information we used, what step happened next, why the result is correct, and what we changed after testing.

Let's Practice

Let's practice with these activities:

Activity 1: Java Around Us Quick Map Type: individual Time: 8 minutes Instruction: Create a quick map showing where programming appears in everyday systems. Student task: - Write Java or Programming at the center. - Add five systems: school portal, ATM, mobile app, game, and store cashier. - For each system, write one possible input and one possible output. Output: Submit a concept map with at least five systems and five input/output pairs.

Activity 2: Input, Process, Output Hunt Type: pair Time: 10 minutes Instruction: Identify IPO parts from familiar systems. Student task: - Choose two systems from your quick map. - Write the input, process, and output for each. - Explain which part a Java program might handle. Output: Submit two IPO rows and one short explanation.

Activity 3: Vocabulary Match Type: individual Time: 7 minutes Instruction: Match each term to the best meaning. Student task: - Program - _ ***- Developer -*** - *Output* - _ ***- Application -*** - Java - ____ Output: Submit completed matches and one sentence using the word program.

Activity 4: Explain Java Simply Type: class discussion Time: 8 minutes Instruction: Explain Java using beginner-friendly language. Student task: - Write a two-sentence explanation. - Use the words instructions and output. - Share with a partner and improve one word or phrase. Output: Submit the improved two-sentence explanation.

Activity 5: Exit Ticket Type: exit ticket Time: 5 minutes Instruction: Show what we can remember before leaving the lesson. Student task: - Write three Java-related terms. - Write one system that could use programming. - Write one question you still have. Output: Submit a 3-1-1 exit ticket.

Now We Apply

Now we apply it through these tasks: 1. Choose one daily system and describe its input, process, and output. 2. Create a mini poster titled Java Around Us with four examples. 3. Write a short paragraph explaining why Java is useful for learning programming.

Challenge Task

Design a one-page Java Concepts Quick Map showing at least eight connected ideas: Java, program, code, compiler, output, error, developer, and user.

How Our Work Will Be Checked

Criteria - 20 points total - Correctness (5): Output or answer follows the required concept and instructions. - Completeness (4): All required parts are present. - Logic and sequence (4): Steps, code, diagram, or pseudocode follow a clear order. - Explanation (3): Student can explain what the work does and why it is correct. - Neatness/readability (2): Work is easy to read, label, and check. - Effort and revision (2): Student improves the work after feedback.

Let's Reflect

Let's reflect: - What concept from this lesson is clearest to you now? - What mistake did you notice or correct during practice? - Which activity helped you understand the lesson best? - How can this skill help you design or write a Java program? - What do you still need to practice before the next lesson?

How We Show Learning

Evidence we will use to check learning: - A concept map with correct connections. - Examples connect Java/programming to real systems. - Student explanations mention instructions, output, or problem solving. - Completed guided-practice work with visible corrections or annotations. - A short explanation connecting the output to the lesson target.

Student Activities

Java Concepts Quick Map: create a concept map with Java at the center. Real-World Java Hunt: list five systems that might use programming logic. Vocabulary Match: match program, developer, application, and output to meanings. Explain It Simply: explain Java to a younger student. Think-Pair-Share: Why should programmers test their work? Exit Ticket: write three Java-related terms learned today. Choose one daily system and describe its input, process, and output. Create a mini poster titled Java Around Us with four examples. Write a short paragraph explaining why Java is useful for learning programming.

Resources / References

JDK, IDE, projector or screenshots, printed worksheets, notebook, sample code snippets, and teacher-created answer guides.

LESSON 2

Lesson 2: Setting Up The Java Development Environment

What We'll Learn

By the end of this lesson, we can: - Differentiate JDK, JRE, JVM, source code, and IDE. - Follow the basic steps for preparing a Java development environment. - Run a simple Java project or verify that Java is installed.

Words We'll Use

- JDK - Java Development Kit used to build Java programs.
- JRE - Java Runtime Environment used to run Java applications.
- JVM - Java Virtual Machine that executes Java bytecode.
- IDE - software used to write and manage code.
- Compiler - tool that translates source code.

Before We Start

If a carpenter needs tools to build a table, what tools does a programmer need to build a Java program?

Let's Understand

Java toolchain guide Let's work through Lesson 2: Setting Up The Java Development Environment together. Our focus is the tools needed to write, compile, run, and test Java programs. We will move from meaning, to examples, to practice, then to a task we can explain. What this lesson means: - Java is a programming language we use to write instructions for a computer. - A Java program is written as source code, saved in a `.java` file, compiled, then run. - The JDK gives us the tools for building Java programs; the JVM runs the compiled bytecode. - An IDE helps us write, organize, run, and debug code in one workspace. - A beginner Java program usually has a class, a main method, statements, braces, and comments. - We read code from the outside container toward the inside instructions, then we trace what will display. How we study it step by step: - First, we connect the lesson to a familiar situation: If a carpenter needs tools to build a table, what tools does a programmer need to build a Java program? - Next, we name the important parts so everyone uses the same vocabulary. - Then, we study JDK, JRE, JVM and ask: what does it do, where do we see it, and how can we check it? - Then, we study IDE setup and ask: what does it do, where do we see it, and how can we check it? - Then, we study Checking Java version and ask: what does it do, where do we see it, and how can we check it? - Then, we study Creating a first project and ask: what does it do, where do we see it, and how can we check it? - After that, we compare a correct example with a common wrong version. - We trace the example slowly before running, drawing, or submitting anything. - Finally, we explain the result in our own words so the activity is not just copying. Rules and patterns we should remember: - Rule: Java is case-sensitive, so `System` and `system` are not the same. Example: `System.out.println("Hi");` works, but `system.out.println("Hi");` causes an error. Check: Which word must start with a capital S? - Rule: Text output must be inside double quotation marks. Example: `System.out.println("Hello, Java!");` displays Hello, Java! Check: What text will appear on the screen? - Rule: Many Java statements end with a semicolon. Example: `int score = 90;` is complete, but `int score = 90` is missing its ending mark. Check: What symbol should we add at the end? - Rule: Braces { and } must be paired because they show where a block starts and ends. Example: `public class Demo { public static void main(String[] args) { } }` has matching class and main braces. Check: How many opening and closing braces can we count? - Rule: Comments explain code for humans and are ignored when the program runs. Example: `// This line prints a greeting` helps us understand the next statement. Check: Will a comment display in the console? Common mistakes we will watch for: - Forgetting capitalization in Java keywords or class names. - Missing quotation marks, semicolons, or closing braces. - Thinking comments run as code. - Changing code without predicting the output first. - Submitting a screenshot without explaining what happened. Mini-check before practice: - What part of the Java program is the container? - Where does the program start running? - What line displays output? - What syntax rule should we check before running? When we move to practice, our goal is not only to get an answer. Our goal is to explain the thinking: what information we used, what step happened next, why the result is correct, and what we changed after testing.

Let's Practice

Let's practice with these activities:

Activity 1: Tool Role Sort Type: individual Time: 8 minutes Instruction: Sort each Java tool by what it does. Student task: - JDK - builds, runs, or writes? - IDE - builds, runs, or writes? - Compiler - translates or displays? - JVM - edits code or runs bytecode? Output: Submit a tool-role table with four correct rows.

Activity 2: Setup Checklist Type: hands-on coding Time: 12 minutes Instruction: Check whether the computer is ready for Java. Student task: - Open the IDE or terminal. - Find the Java version or IDE welcome screen. - Record what tool opened successfully. - Write one issue if something did not open. Output: Submit a setup checklist and screenshot or written evidence.

Activity 3: First Project Test Run Type: hands-on coding Time: 15 minutes Instruction: Create a basic Java project and run a simple output program. Student task: - Create a new Java file. - Type the sample program. - Run the program. - Circle or copy the console output. Code / model:

```
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, CC 102!");
    }
}
```

Output: Submit the output and one sentence explaining what ran successfully.

Activity 4: Troubleshooting Log Type: pair Time: 8 minutes Instruction: Practice reading setup or run problems calmly. Student task: - Write one problem that could happen during setup. - Write one possible cause. - Write one possible fix. - Ask a partner if the fix is clear. Output: Submit one problem-cause-fix row.

Activity 5: Exit Ticket Type: exit ticket Time: 5 minutes Instruction: Check tool understanding. Student task: - Which tool helps us write code? - Which tool runs bytecode? - Which command or screen proved Java worked? Output: Submit three short answers.

Now We Apply

Now we apply it through these tasks: 1. Create a setup checklist for a classmate. 2. Run the Hello World sample and write what happened. 3. Label a diagram showing source code, compiler, JVM, and output.

Challenge Task

Produce a Java Setup Confirmation Sheet with tool names, installation/status notes, a test-run result, and one troubleshooting tip.

How Our Work Will Be Checked

Criteria - 20 points total - Correctness (5): Output or answer follows the required concept and instructions. - Completeness (4): All required parts are present. - Logic and sequence (4): Steps, code, diagram, or pseudocode follow a clear order. - Explanation (3): Student can explain what the work does and why it is correct. - Neatness/readability (2): Work is easy to read, label, and check. - Effort and revision (2): Student improves the work after feedback.

Let's Reflect

Let's reflect: - What concept from this lesson is clearest to you now? - What mistake did you notice or correct during practice? - Which activity helped you understand the lesson best? - How can this skill help you design or write a Java program? - What do you still need to practice before the next lesson?

How We Show Learning

Evidence we will use to check learning: - Students identify JDK and IDE roles. - A successful run displays simple output. - Setup checklist is ordered and practical. - Completed guided-practice work with visible corrections or annotations. - A short explanation connecting the output to the lesson target.

Student Activities

Java Tools Setup Check: identify whether JDK and IDE are installed. Version Check: record the Java version or IDE welcome screen. Tool Match: match JDK, IDE, compiler, and output window. First Project Test Run: create a new Java project and run a sample program. Screenshot Evidence: capture proof that Java ran successfully. Troubleshooting Log: list one setup issue and solution. Create a setup checklist for a classmate. Run the Hello World sample and write what happened. Label a diagram showing source code, compiler, JVM, and output.

Resources / References

JDK, IDE, projector or screenshots, printed worksheets, notebook, sample code snippets, and teacher-created answer guides.

LESSON 3

Lesson 3: Writing And Running A First Java Program

What We'll Learn

By the end of this lesson, we can: - Type and run a simple Java program. - Explain what the Hello World program displays. - Modify a print statement to show personalized output.

Words We'll Use

- Class - a named container for Java code.
- Main method - the starting point of a Java program.
- Statement - a single instruction.
- String - text enclosed in double quotation marks.
- Output - information displayed by a program.

Before We Start

What message would you like your first program to display, and why?

Let's Understand

Java program structure guide Let's work through Lesson 3: Writing And Running A First Java Program together. Our focus is writing, running, and explaining a first Java output program. We will move from meaning, to examples, to practice, then to a task we can explain. What this lesson means: - Java is a programming language we use to write instructions for a computer. - A Java program is written as source code, saved in a `.java` file, compiled, then run. - The JDK gives us the tools for building Java programs; the JVM runs the compiled bytecode. - An IDE helps us write, organize, run, and debug code in one workspace. - A beginner Java program usually has a class, a main method, statements, braces, and comments. - We read code from the outside container toward the inside instructions, then we trace what will display. How we study it step by step: - First, we connect the lesson to a familiar situation: What message would you like your first program to display, and why? - Next, we name the important parts so everyone uses the same vocabulary. - Then, we study Hello World and ask: what does it do, where do we see it, and how can we check it? - Then, we study Print statements and ask: what does it do, where do we see it, and how can we check it? - Then, we study Program output and ask: what does it do, where do we see it, and how can we check it? - Then, we study Simple modification and ask: what does it do, where do we see it, and how can we check it? - After that, we compare a correct example with a common wrong version. - We trace the example slowly before running, drawing, or submitting anything. - Finally, we explain the result in our own words so the activity is not just copying. Rules and patterns we should remember: - Rule: Java is case-sensitive, so `System` and `system` are not the same. Example: `System.out.println("Hi");` works, but `system.out.println("Hi");` causes an error. Check: Which word must start with a capital S? - Rule: Text output must be inside double quotation marks. Example: `System.out.println("Hello, Java!");` displays Hello, Java! Check: What text will appear on the screen? - Rule: Many Java statements end with a semicolon. Example: `int score = 90;` is complete, but `int score = 90` is missing its ending mark. Check: What symbol should we add at the end? - Rule: Braces { and } must be paired because they show where a block starts and ends. Example: `public class Demo { public static void main(String[] args) { } }` has matching class and main braces. Check: How many opening and closing braces can we count? - Rule: Comments explain code for humans and are ignored when the program runs. Example: `// This line prints a greeting` helps us understand the next statement. Check: Will a comment display in the console? Common mistakes we will watch for: - Forgetting capitalization in Java keywords or class names. - Missing quotation marks, semicolons, or closing braces. - Thinking comments run as code. - Changing code without predicting the output first. - Submitting a screenshot without explaining what happened. Mini-check before practice: - What part of the Java program is the container? - Where does the program start running? - What line displays output? - What syntax rule should we check before running? When we move to practice, our goal is not only to get an answer. Our goal is to explain the thinking: what information we used, what step happened next, why the result is correct, and what we changed after testing.

Let's Practice

Let's practice with these activities:

Activity 1: Predict The Output Type: individual Time: 6 minutes Instruction: Read the Java code before running it and predict what appears. Student task: - Underline the output statement. - Write the exact text you think will appear. - Run or compare with the model output. Code / model:

```
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, CC 102!");
    }
}
```

Output: Submit prediction, actual output, and one correction if needed.

Activity 2: Change The Message Type: hands-on coding Time: 10 minutes Instruction: Edit the printed message while keeping valid Java syntax. Student task: - Change Hello, CC 102! to your name. - Keep the quotation marks. - Keep the semicolon. - Run the program again. Output: Submit the edited print line and output.

Activity 3: Add More Lines Type: hands-on coding Time: 10 minutes Instruction: Add more output statements to create a short profile. Student task: - Add one line for your section. - Add one line for your learning goal. - Predict the order of output lines before running. Output: Submit three output lines in the correct order.

Activity 4: Mini Debug Type: pair Time: 8 minutes Instruction: Find and fix the syntax error. Student task: - Find the missing symbol. - Rewrite the corrected line. - Explain why Java needs the correction. Code / model:

```
System.out.println("Welcome to Java!");
System.out.println("We are learning output")
```

Output: Submit corrected code and one explanation sentence.

Activity 5: Exit Ticket Type: exit ticket Time: 5 minutes Instruction: Summarize the first Java program. Student task: - What line displays output? - What symbols surround text? - What symbol ends the statement? Output: Submit three short answers.

Now We Apply

Now we apply it through these tasks: 1. Create a program that prints your name, course, and one goal. 2. Write three output statements for a class announcement. 3. Submit a screenshot or written copy of your output.

Challenge Task

Create a Welcome Program with at least five output lines: greeting, name, course, learning goal, and motivational message. Explain each line in one sentence.

How Our Work Will Be Checked

Criteria - 20 points total - Correctness (5): Output or answer follows the required concept and instructions. - Completeness (4): All required parts are present. - Logic and sequence (4): Steps, code, diagram, or pseudocode follow a clear order. - Explanation (3): Student can explain what the work does and why it is correct. - Neatness/readability (2): Work is easy to read, label, and check. - Effort and revision (2): Student improves the work after feedback.

Let's Reflect

Let's reflect: - What concept from this lesson is clearest to you now? - What mistake did you notice or correct during practice? - Which activity helped you understand the lesson best? - How can this skill help you design or write a Java program? - What do you still need to practice before the next lesson?

How We Show Learning

Evidence we will use to check learning: - Program runs without syntax errors. - Output matches required lines. - Student explains that println displays text. - Completed guided-practice work with visible corrections or annotations. - A short explanation connecting the output to the lesson target.

Student Activities

Copy and run the Hello World program. Change the output message to your name and section. Add a second output line with your favorite subject. Predict the output before running the program. Compare print and println behavior. Mini Debug: fix a missing quotation mark. Create a program that prints your name, course, and one goal. Write three output statements for a class announcement. Submit a screenshot or written copy of your output.

Resources / References

JDK, IDE, projector or screenshots, printed worksheets, notebook, sample code snippets, and teacher-created answer guides.

LESSON 4

Lesson 4: Java Program Structure And Code Comments

What We'll Learn

By the end of this lesson, we can: - Identify class name, main method, statements, comments, and variables. - Explain the purpose of each Java program part. - Add comments to describe what code does.

Words We'll Use

- Comment - note in code ignored by Java.
- Variable - named storage for a value.
- Method - block of code that performs a task.
- Brace - curly symbol used to group code blocks.
- Identifier - name used for classes, variables, or methods.

Before We Start

Which parts of a short Java program look like names, actions, notes, or containers?

Let's Understand

Java toolchain guide Let's work through Lesson 4: Java Program Structure And Code Comments together. Our focus is labeling and explaining the basic structure of a Java program. We will move from meaning, to examples, to practice, then to a task we can explain. What this lesson means: - Java is a programming language we use to write instructions for a computer. - A Java program is written as source code, saved in a `.java` file, compiled, then run. - The JDK gives us the tools for building Java programs; the JVM runs the compiled bytecode. - An IDE helps us write, organize, run, and debug code in one workspace. - A beginner Java program usually has a class, a main method, statements, braces, and comments. - We read code from the outside container toward the inside instructions, then we trace what will display. How we study it step by step: - First, we connect the lesson to a familiar situation: Which parts of a short Java program look like names, actions, notes, or containers? - Next, we name the important parts so everyone uses the same vocabulary. - Then, we study Class declaration and ask: what does it do, where do we see it, and how can we check it? - Then, we study Main method and ask: what does it do, where do we see it, and how can we check it? - Then, we study Statements and ask: what does it do, where do we see it, and how can we check it? - Then, we study Comments and ask: what does it do, where do we see it, and how can we check it? - Then, we study Variables and ask: what does it do, where do we see it, and how can we check it? - After that, we compare a correct example with a common wrong version. - We trace the example slowly before running, drawing, or submitting anything. - Finally, we explain the result in our own words so the activity is not just copying. Rules and patterns we should remember: - Rule: Java is case-sensitive, so `System` and `system` are not the same. Example: `System.out.println("Hi");` works, but `system.out.println("Hi");` causes an error. Check: Which word must start with a capital S? - Rule: Text output must be inside double quotation marks. Example: `System.out.println("Hello, Java!");` displays Hello, Java! Check: What text will appear on the screen? - Rule: Many Java statements end with a semicolon. Example: `int score = 90;` is complete, but `int score = 90` is missing its ending mark. Check: What symbol should we add at the end? - Rule: Braces `{` and `}` must be paired because they show where a block starts and ends. Example: `public class Demo { public static void main(String[] args) { } }` has matching class and main braces. Check: How many opening and closing braces can we count? - Rule: Comments explain code for humans and are ignored when the program runs. Example: `// This line prints a greeting` helps us understand the next statement. Check: Will a comment display in the console? Common mistakes we will watch for: - Forgetting capitalization in Java keywords or class names. - Missing quotation marks, semicolons, or closing braces. - Thinking comments run as code. - Changing code without predicting the output first. - Submitting a screenshot without explaining what happened. Mini-check before practice: - What part of the Java program is the container? - Where does the program start running? - What line displays output? - What syntax rule should we check before running? When we move to practice, our goal is not only to get an answer. Our goal is to explain the thinking: what information we used, what step happened next, why the result is correct, and what we changed after testing.

Let's Practice

Let's practice with these activities:

Activity 1: Label The Program Type: individual Time: 10 minutes Instruction: Identify the main parts of a Java program. Student task: - Label the class name. - Label the main method. - Label two variables. - Label one output statement. - Label one comment. Code / model:

```
public class StudentInfo {
    public static void main(String[] args) {
        String name = "Ana";
        int age = 18;
        double grade = 95.5;
        System.out.println(name + " is " + age + " years old.");
        System.out.println("Grade: " + grade);
    }
}
```

Output: Submit a labeled copy or a table of line number and program part.

Activity 2: Brace Pair Check Type: pair Time: 7 minutes Instruction: Trace which braces belong together. Student task: - Count opening braces. - Count closing braces. - Draw pairs or write Class block and Main block. - Explain what happens if one brace is missing. Output: Submit brace count and one explanation.

Activity 3: Comment The Code Type: hands-on coding Time: 10 minutes Instruction: Add helpful comments that explain code purpose. Student task: - Add one comment before the variable declarations. - Add one comment before output statements. - Do not comment every single word. - Run the code to confirm comments do not display. Output: Submit code with two meaningful comments and output.

Activity 4: Variable Purpose Table Type: individual Time: 8 minutes Instruction: Explain what each variable stores. Student task: - Create columns: variable, data type, value, purpose. - Fill in name, age, and grade. - Write one sentence explaining why variables are useful. Output: Submit the table and sentence.

Activity 5: Exit Ticket Type: exit ticket Time: 5 minutes Instruction: Check program-part vocabulary. Student task: - What part starts the Java program? - What part stores a value? - What part is ignored by Java but helps humans? Output: Submit three answers.

Now We Apply

Now we apply it through these tasks: 1. Write a Java program with at least two variables and two comments. 2. Create a table: Java Part and Purpose. 3. Exchange code and label each other's program.

Challenge Task

Submit an annotated Java program with class, main method, three variables, three output statements, and comments explaining each section.

How Our Work Will Be Checked

Criteria - 20 points total - Correctness (5): Output or answer follows the required concept and instructions. - Completeness (4): All required parts are present. - Logic and sequence (4): Steps, code, diagram, or pseudocode follow a clear order. - Explanation (3): Student can explain what the work does and why it is correct. - Neatness/readability (2): Work is easy to read, label, and check. - Effort and revision (2): Student improves the work after feedback.

Let's Reflect

Let's reflect: - What concept from this lesson is clearest to you now? - What mistake did you notice or correct during practice? - Which activity helped you understand the lesson best? - How can this skill help you design or write a Java program? - What do you still need to practice before the next lesson?

How We Show Learning

Evidence we will use to check learning: - Labels correctly identify class, main, variables, comments, statements. - Comments explain purpose, not just repeat code. - Annotated program is readable. - Completed guided-practice work with visible corrections or annotations. - A short explanation connecting the output to the lesson target.

Student Activities

Java Program Walk-Through: label class, main, statements, comments, variables. Color Code the Program. Comment the Code: add comments to explain three lines. Parts Matching: match Java part to its purpose. Explain to a Partner: describe what a variable stores. Code Labeling Quiz. Write a Java program with at least two variables and two comments. Create a table: Java Part and Purpose. Exchange code and label each other's program.

Resources / References

JDK, IDE, projector or screenshots, printed worksheets, notebook, sample code snippets, and teacher-created answer guides.

LESSON 5

Lesson 5: Java Syntax Rules And Debugging Basics

What We'll Learn

By the end of this lesson, we can: - Identify common Java syntax errors. - Correct missing semicolons, brackets, quotation marks, and wrong keywords. - Explain why syntax rules matter.

Words We'll Use

- Syntax - rules for writing valid code.
- Error - problem that prevents or affects execution.
- Semicolon - symbol that ends many Java statements.
- Brace - symbol used to group code.
- Debugging - finding and fixing errors.

Before We Start

Why do sentences need punctuation? How is punctuation similar to syntax in programming?

Let's Understand

Java syntax correction guide Let's work through Lesson 5: Java Syntax Rules And Debugging Basics together. Our focus is recognizing Java syntax rules and correcting beginner errors. We will move from meaning, to examples, to practice, then to a task we can explain. What this lesson means: - Java is a programming language we use to write instructions for a computer. - A Java program is written as source code, saved in a `.java` file, compiled, then run. - The JDK gives us the tools for building Java programs; the JVM runs the compiled bytecode. - An IDE helps us write, organize, run, and debug code in one workspace. - A beginner Java program usually has a class, a main method, statements, braces, and comments. - We read code from the outside container toward the inside instructions, then we trace what will display. How we study it step by step: - First, we connect the lesson to a familiar situation: Why do sentences need punctuation? How is punctuation similar to syntax in programming? - Next, we name the important parts so everyone uses the same vocabulary. - Then, we study Semicolons and ask: what does it do, where do we see it, and how can we check it? - Then, we study Quotation marks and ask: what does it do, where do we see it, and how can we check it? - Then, we study Braces and ask: what does it do, where do we see it, and how can we check it? - Then, we study Capitalization and ask: what does it do, where do we see it, and how can we check it? - Then, we study Debugging and ask: what does it do, where do we see it, and how can we check it? - After that, we compare a correct example with a common wrong version. - We trace the example slowly before running, drawing, or submitting anything. - Finally, we explain the result in our own words so the activity is not just copying. Rules and patterns we should remember: - Rule: Java is case-sensitive, so `System` and `system` are not the same. Example: `System.out.println("Hi");` works, but `system.out.println("Hi");` causes an error. Check: Which word must start with a capital S? - Rule: Text output must be inside double quotation marks. Example: `System.out.println("Hello, Java!");` displays Hello, Java! Check: What text will appear on the screen? - Rule: Many Java statements end with a semicolon. Example: `int score = 90;` is complete, but `int score = 90` is missing its ending mark. Check: What symbol should we add at the end? - Rule: Braces { and } must be paired because they show where a block starts and ends. Example: `public class Demo { public static void main(String[] args) { } }` has matching class and main braces. Check: How many opening and closing braces can we count? - Rule: Comments explain code for humans and are ignored when the program runs. Example: `// This line prints a greeting` helps us understand the next statement. Check: Will a comment display in the console? Common mistakes we will watch for: - Forgetting capitalization in Java keywords or class names. - Missing quotation marks, semicolons, or closing braces. - Thinking comments run as code. - Changing code without predicting the output first. - Submitting a screenshot without explaining what happened. Mini-check before practice: - What part of the Java program is the container? - Where does the program start running? - What line displays output? - What syntax rule should we check before running? When we move to practice, our goal is not only to get an answer. Our goal is to explain the thinking: what information we used, what step happened next, why the result is correct, and what we changed after testing.

Let's Practice

Let's practice with these activities:

Activity 1: Syntax Spot-The-Error Type: individual Time: 10 minutes Instruction: Find beginner Java syntax mistakes. Student task: - Check capitalization. - Check quotation marks. - Check semicolons. - Check braces. - Circle or list each mistake. Code / model:

```
public class SyntaxCheck {
    public static void main(String[] args) {
        system.out.println("Hi)
        System.out.println("Java")
    }
}
```

Output: Submit the list of errors and corrected code.

Activity 2: Fix One Line At A Time Type: pair Time: 10 minutes Instruction: Correct each broken line and explain the rule. Student task: - Fix `System.out.println("Hello")`. - Fix `system.out.println("Hi");`. - Fix `System.out.println(Hi);`. - Write the rule used for each fix. Output: Submit three corrected lines and three matching rules.

Activity 3: Error Explanation Type: individual Time: 8 minutes Instruction: Explain why syntax matters. Student task: - Choose one corrected error. - Write what was wrong. - Write how the correction helps Java understand the instruction. Output: Submit a three-sentence explanation.

Activity 4: Create An Error For A Partner Type: pair Time: 10 minutes Instruction: Write one intentionally broken Java line for a partner to fix. Student task: - Create one missing semicolon error or quote error. - Trade with a partner. - Fix your partner's line. - Explain the correction. Output: Submit the broken line, corrected line, and explanation.

Activity 5: Exit Ticket Type: exit ticket Time: 5 minutes Instruction: Use the syntax checklist before leaving. Student task: - Write one capitalization rule. - Write one punctuation rule. - Write one brace rule. Output: Submit the three-rule checklist.

Now We Apply

Now we apply it through these tasks: 1. Correct a broken Hello World program with at least five errors. 2. Make a personal Java Syntax Checklist. 3. Write three beginner syntax rules.

Challenge Task

Debug a short Java program with at least eight errors. Submit corrected code and a correction log.

How Our Work Will Be Checked

Criteria - 20 points total - Correctness (5): Output or answer follows the required concept and instructions. - Completeness (4): All required parts are present. - Logic and sequence (4): Steps, code, diagram, or pseudocode follow a clear order. - Explanation (3): Student can explain what the work does and why it is correct. - Neatness/readability (2): Work is easy to read, label, and check. - Effort and revision (2): Student improves the work after feedback.

Let's Reflect

Let's reflect: - What concept from this lesson is clearest to you now? - What mistake did you notice or correct during practice? - Which activity helped you understand the lesson best? - How can this skill help you design or write a Java program? - What do you still need to practice before the next lesson?

How We Show Learning

Evidence we will use to check learning: - Common syntax errors are found and corrected. - Correction log explains each fix. - Corrected code runs or is logically valid. - Completed guided-practice work with visible corrections or annotations. - A short explanation connecting the output to the lesson target.

Student Activities

Syntax Spot-the-Error: circle mistakes in five samples. Fix the Error Race: correct semicolons and braces. Capitalization Check. Quotation Mark Repair. Error Explanation: write why each correction is needed. Create an Error: write one wrong line for a partner to fix. Correct a broken Hello World program with at least five errors. Make a personal Java Syntax Checklist. Write three beginner syntax rules.

Resources / References

JDK, IDE, projector or screenshots, printed worksheets, notebook, sample code snippets, and teacher-created answer guides.

LESSON 6

Lesson 6: Algorithmic Thinking With Everyday Tasks

What We'll Learn

By the end of this lesson, we can: - Define algorithm using daily-life examples. - Arrange steps in correct order. - Explain why order matters.

Words We'll Use

- Algorithm - ordered steps to solve a problem.
- Step - one action in a process.
- Sequence - correct order.
- Input - information needed.
- Output - result after steps.

Before We Start

Describe how you log in to a computer. What happens if one step is skipped?

Let's Understand

Algorithm and IPO guide Let's work through Lesson 6: Algorithmic Thinking With Everyday Tasks together. Our focus is algorithms as ordered steps for completing familiar tasks. We will move from meaning, to examples, to practice, then to a task we can explain. What this lesson means: - An algorithm is an ordered list of steps that solves a problem or completes a task. - Before we code, we identify the input, process, and output so our solution has direction. - Good steps are clear enough that another person can follow them without guessing. - The order matters because computers follow instructions exactly as written. - We can test an algorithm with sample values before writing Java code. - A strong algorithm is specific, complete, logical, and easy to revise. How we study it step by step: - First, we connect the lesson to a familiar situation: Describe how you log in to a computer. What happens if one step is skipped? - Next, we name the important parts so everyone uses the same vocabulary. - Then, we study Daily algorithms and ask: what does it do, where do we see it, and how can we check it? - Then, we study Order and sequence and ask: what does it do, where do we see it, and how can we check it? - Then, we study Input and output and ask: what does it do, where do we see it, and how can we check it? - After that, we compare a correct example with a common wrong version. - We trace the example slowly before running, drawing, or submitting anything. - Finally, we explain the result in our own words so the activity is not just copying. Rules and patterns we should remember: - Rule: Use action words such as read, compute, compare, display, repeat, or stop. Example: Step 1: Read firstNumber. Step 2: Read secondNumber. Step 3: Compute sum. Check: Which word tells us the action in Step 3? - Rule: Put one clear action per step when possible. Example: Better: `Read grade`, then `Compare grade to 75`; unclear: `Check everything`. Check: Which version can a beginner follow? - Rule: Include the needed input before the process step. Example: For sum of two numbers, we first need number1 and number2 before computing sum. Check: What information must we ask for first? - Rule: Include the output after the process or decision step. Example: After computing `sum = 5 + 7`, we display `12`. Check: What should the user see at the end? - Rule: Test the algorithm with at least one sample value. Example: If number1 is 5 and number2 is 7, the expected sum is 12. Check: Does our step list produce 12? Common mistakes we will watch for: - Writing a paragraph instead of ordered steps. - Starting with computation before identifying inputs. - Skipping the output step. - Combining several actions into one vague step. - Not testing whether another person can follow the algorithm. Mini-check before practice: - What is our input? - What process happens to the input? - What output should appear? - Are the steps clear enough for a classmate to follow? When we move to practice, our goal is not only to get an answer. Our goal is to explain the thinking: what information we used, what step happened next, why the result is correct, and what we changed after testing.

Let's Practice

Let's practice with these activities:

Activity 1: Everyday Steps To Algorithm Type: individual Time: 8 minutes Instruction: Turn a familiar task into ordered steps. Student task: - Choose logging in to a computer or submitting an assignment. - Write 8-10 numbered steps. - Mark the input/materials and expected output. Output: Submit an ordered algorithm with input/materials and output.

Activity 2: Arrange The Algorithm Type: pair Time: 8 minutes Instruction: Put scrambled steps in the correct order. Student task: - Arrange: display welcome screen, type password, click login, open computer, type username. - Rewrite the corrected order. - Explain why one step must come first. Output: Submit corrected order and one explanation.

Activity 3: Missing Step Detective Type: pair Time: 8 minutes Instruction: Find missing steps that make an algorithm unclear. Student task: - Read: Open IDE, run program, submit output. - Add at least three missing steps. - Check if a beginner could follow it. Output: Submit improved algorithm.

Activity 4: Partner Test Type: pair Time: 10 minutes Instruction: Test whether another person can follow your steps exactly. Student task: - Exchange algorithms. - Follow your partner's steps without adding hidden steps. - Mark confusing steps. - Revise your own algorithm. Output: Submit original steps, partner feedback, and revised steps.

Activity 5: Exit Ticket Type: exit ticket Time: 5 minutes Instruction: Check algorithm vocabulary. Student task: - Define algorithm in your own words. - Write one reason order matters. - Write one example of output. Output: Submit three short answers.

Now We Apply

Now we apply it through these tasks: 1. Write an algorithm for submitting an online assignment. 2. Create an algorithm for preparing your Java workspace. 3. Revise your algorithm after partner feedback.

Challenge Task

Create a clear 10-step algorithm for a daily school or computer task with title, input/materials, ordered steps, expected output, and one safety note.

How Our Work Will Be Checked

Criteria - 20 points total - Correctness (5): Output or answer follows the required concept and instructions. - Completeness (4): All required parts are present. - Logic and sequence (4): Steps, code, diagram, or pseudocode follow a clear order. - Explanation (3): Student can explain what the work does and why it is correct. - Neatness/readability (2): Work is easy to read, label, and check. - Effort and revision (2): Student improves the work after feedback.

Let's Reflect

Let's reflect: - What concept from this lesson is clearest to you now? - What mistake did you notice or correct during practice? - Which activity helped you understand the lesson best? - How can this skill help you design or write a Java program? - What do you still need to practice before the next lesson?

How We Show Learning

Evidence we will use to check learning: - Steps are ordered and specific. - Algorithm can be followed by another person. - Output/result is clear. - Completed guided-practice work with visible corrections or annotations. - A short explanation connecting the output to the lesson target.

Student Activities

Everyday Steps to Algorithm. Arrange the Algorithm. Missing Step Detective. Too Broad or Clear. Partner Test. Reflection Check. Write an algorithm for submitting an online assignment. Create an algorithm for preparing your Java workspace. Revise your algorithm after partner feedback.

Resources / References

JDK, IDE, projector or screenshots, printed worksheets, notebook, sample code snippets, and teacher-created answer guides.

LESSON 7

Lesson 7: Writing Algorithms With Input, Process, And Output

What We'll Learn

By the end of this lesson, we can: - Identify input, process, and output. - Write ordered algorithm steps. - Revise vague steps into specific steps.

Words We'll Use

- Problem - task that needs a solution.
- Input - values needed.
- Process - computation performed.
- Output - final result.
- Revision - improving after checking.

Before We Start

If the problem is display the sum of two numbers, what information must the program know first?

Let's Understand

Algorithm and IPO guide Let's work through Lesson 7: Writing Algorithms With Input, Process, And Output together. Our focus is turning simple programming problems into clear ordered solutions. We will move from meaning, to examples, to practice, then to a task we can explain. What this lesson means: - An algorithm is an ordered list of steps that solves a problem or completes a task. - Before we code, we identify the input, process, and output so our solution has direction. - Good steps are clear enough that another person can follow them without guessing. - The order matters because computers follow instructions exactly as written. - We can test an algorithm with sample values before writing Java code. - A strong algorithm is specific, complete, logical, and easy to revise. How we study it step by step: - First, we connect the lesson to a familiar situation: If the problem is display the sum of two numbers, what information must the program know first? - Next, we name the important parts so everyone uses the same vocabulary. - Then, we study IPO and ask: what does it do, where do we see it, and how can we check it? - Then, we study Problem analysis and ask: what does it do, where do we see it, and how can we check it? - Then, we study Step writing and ask: what does it do, where do we see it, and how can we check it? - After that, we compare a correct example with a common wrong version. - We trace the example slowly before running, drawing, or submitting anything. - Finally, we explain the result in our own words so the activity is not just copying. Rules and patterns we should remember: - Rule: Use action words such as read, compute, compare, display, repeat, or stop. Example: Step 1: Read firstNumber. Step 2: Read secondNumber. Step 3: Compute sum. Check: Which word tells us the action in Step 3? - Rule: Put one clear action per step when possible. Example: Better: `Read grade`, then `Compare grade to 75`; unclear: `Check everything`. Check: Which version can a beginner follow? - Rule: Include the needed input before the process step. Example: For sum of two numbers, we first need number1 and number2 before computing sum. Check: What information must we ask for first? - Rule: Include the output after the process or decision step. Example: After computing `sum = 5 + 7`, we display `12`. Check: What should the user see at the end? - Rule: Test the algorithm with at least one sample value. Example: If number1 is 5 and number2 is 7, the expected sum is 12. Check: Does our step list produce 12? Common mistakes we will watch for: - Writing a paragraph instead of ordered steps. - Starting with computation before identifying inputs. - Skipping the output step. - Combining several actions into one vague step. - Not testing whether another person can follow the algorithm. Mini-check before practice: - What is our input? - What process happens to the input? - What output should appear? - Are the steps clear enough for a classmate to follow? When we move to practice, our goal is not only to get an answer. Our goal is to explain the thinking: what information we used, what step happened next, why the result is correct, and what we changed after testing.

Let's Practice

Let's practice with these activities:

Activity 1: IPO Table Type: individual Time: 8 minutes Instruction: Identify input, process, and output before writing steps. Student task: - Problem: display the sum of two numbers. - Input: ***_ and*** *.* - *Process:* ***_. - Output:*** . Output: Submit the completed IPO table.

Activity 2: Write The Algorithm Type: individual Time: 10 minutes Instruction: Write clear steps for the sum problem. Student task: - Start. - Read first number. - Read second number. - Compute sum. - Display sum. - End. Rewrite these as complete numbered steps. Output: Submit a complete algorithm.

Activity 3: Trace With Values Type: pair Time: 8 minutes Instruction: Test the algorithm with sample values. Student task: - Use first number = 8 and second number = 4. - Write the value after the process step. - Write the output. - Try another pair of numbers. Output: Submit two trace rows.

Activity 4: Improve Vague Steps Type: pair Time: 8 minutes Instruction: Rewrite unclear algorithm steps. Student task: - Improve: Get numbers. - Improve: Do math. - Improve: Show answer. - Explain why the new steps are clearer. Output: Submit three improved steps and one explanation.

Activity 5: Exit Ticket Type: exit ticket Time: 5 minutes Instruction: Review IPO planning. Student task: - What is input? - What is process? - What is output? Output: Submit three definitions with examples.

Now We Apply

Now we apply it through these tasks: 1. Write algorithms for area, total price, and pass/fail grade. 2. Explain one algorithm to a partner. 3. Create an IPO table before writing.

Challenge Task

Write three complete algorithms: sum of two numbers, larger of two numbers, and average of three grades. Each includes IPO and ordered steps.

How Our Work Will Be Checked

Criteria - 20 points total - Correctness (5): Output or answer follows the required concept and instructions. - Completeness (4): All required parts are present. - Logic and sequence (4): Steps, code, diagram, or pseudocode follow a clear order. - Explanation (3): Student can explain what the work does and why it is correct. - Neatness/readability (2): Work is easy to read, label, and check. - Effort and revision (2): Student improves the work after feedback.

Let's Reflect

Let's reflect: - What concept from this lesson is clearest to you now? - What mistake did you notice or correct during practice? - Which activity helped you understand the lesson best? - How can this skill help you design or write a Java program? - What do you still need to practice before the next lesson?

How We Show Learning

Evidence we will use to check learning: - Each algorithm includes input, process, output. - Steps are logical and complete. - Student explains calculation or decision. - Completed guided-practice work with visible corrections or annotations. - A short explanation connecting the output to the lesson target.

Student Activities

IPO Table. Write the Algorithm. Arrange the Algorithm. Missing Step Detective. Peer Review. Improve the Algorithm. Write algorithms for area, total price, and pass/fail grade. Explain one algorithm to a partner. Create an IPO table before writing.

Resources / References

JDK, IDE, projector or screenshots, printed worksheets, notebook, sample code snippets, and teacher-created answer guides.

LESSON 8

Lesson 8: Designing Algorithms For Simple Programming Problems

What We'll Learn

By the end of this lesson, we can: - Analyze a problem statement. - Create an algorithm that solves the problem. - Test an algorithm using sample values.

Words We'll Use

- Problem statement - description of what must be solved.
- Test data - sample values used to check.
- Logic - reasoning connecting steps.
- Trace - manually follow steps.
- Efficiency - solving without unnecessary steps.

Before We Start

How do you know if your algorithm works before coding it?

Let's Understand

Algorithm and IPO guide Let's work through Lesson 8: Designing Algorithms For Simple Programming Problems together. Our focus is creating algorithms independently from given problem statements. We will move from meaning, to examples, to practice, then to a task we can explain. What this lesson means: - An algorithm is an ordered list of steps that solves a problem or completes a task. - Before we code, we identify the input, process, and output so our solution has direction. - Good steps are clear enough that another person can follow them without guessing. - The order matters because computers follow instructions exactly as written. - We can test an algorithm with sample values before writing Java code. - A strong algorithm is specific, complete, logical, and easy to revise. How we study it step by step: - First, we read the problem statement and underline the exact result the program must produce. - Next, we list the required input values before writing any solution steps. - Then, we choose the process: compute, compare, count, convert, or decide. - After that, we write ordered steps using action words that another student can follow. - We test the algorithm with at least two sample inputs, including one boundary or special case. - Finally, we revise any vague step until the expected output is clear before coding. Rules and patterns we should remember: - Rule: Use action words such as read, compute, compare, display, repeat, or stop. Example: Step 1: Read firstNumber. Step 2: Read secondNumber. Step 3: Compute sum. Check: Which word tells us the action in Step 3? - Rule: Put one clear action per step when possible. Example: Better: `Read grade`, then `Compare grade to 75`; unclear: `Check everything`. Check: Which version can a beginner follow? - Rule: Include the needed input before the process step. Example: For sum of two numbers, we first need number1 and number2 before computing sum. Check: What information must we ask for first? - Rule: Include the output after the process or decision step. Example: After computing `sum = 5 + 7`, we display `12`. Check: What should the user see at the end? - Rule: Test the algorithm with at least one sample value. Example: If number1 is 5 and number2 is 7, the expected sum is 12. Check: Does our step list produce 12? Common mistakes we will watch for: - Writing a paragraph instead of ordered steps. - Starting with computation before identifying inputs. - Skipping the output step. - Combining several actions into one vague step. - Not testing whether another person can follow the algorithm. Mini-check before practice: - What is our input? - What process happens to the input? - What output should appear? - Are the steps clear enough for a classmate to follow? When we move to practice, our goal is not only to get an answer. Our goal is to explain the thinking: what information we used, what step happened next, why the result is correct, and what we changed after testing.

Let's Practice

Let's practice with these activities:

Activity 1: Problem Analysis Type: individual Time: 8 minutes Instruction: Read a problem and identify what must be solved. Student task: - Problem: display the larger of two numbers. - Underline the needed input. - Write the decision question. - Write the expected output. Output: Submit problem notes with input, decision, and output.

Activity 2: Build The Algorithm Type: individual Time: 12 minutes Instruction: Create a complete algorithm for a decision problem. Student task: - Start. - Read num1 and num2. - Compare num1 and num2. - Display the larger number. - End. - Add clear Yes/No or if/else wording. Output: Submit the complete larger-number algorithm.

Activity 3: Trace Table Type: pair Time: 10 minutes Instruction: Test the algorithm using sample values. Student task: - Trace num1 = 8, num2 = 3. - Trace num1 = 4, num2 = 9. - Trace num1 = 6, num2 = 6. - Write what should display each time. Output: Submit three trace rows.

Activity 4: Peer Debug Type: pair Time: 10 minutes Instruction: Review a partner's algorithm for missing or unclear logic. Student task: - Check if input is complete. - Check if the decision is clear. - Check if every path has output. - Suggest one revision. Output: Submit peer feedback and revised algorithm.

Activity 5: Exit Ticket Type: exit ticket Time: 5 minutes Instruction: Explain testing before coding. Student task: - Why do we trace an algorithm? - What sample values did we use? - What did we revise? Output: Submit three short answers.

Now We Apply

Now we apply it through these tasks: 1. Create algorithms for total fare, change from payment, and class average. 2. Write sample test data. 3. Revise one algorithm after tracing.

Challenge Task

Select one real-life numerical problem and write IPO, ordered steps, sample test data, trace, and expected output.

How Our Work Will Be Checked

Criteria - 20 points total - Correctness (5): Output or answer follows the required concept and instructions. - Completeness (4): All required parts are present. - Logic and sequence (4): Steps, code, diagram, or pseudocode follow a clear order. - Explanation (3): Student can explain what the work does and why it is correct. - Neatness/readability (2): Work is easy to read, label, and check. - Effort and revision (2): Student improves the work after feedback.

Let's Reflect

Let's reflect: - What concept from this lesson is clearest to you now? - What mistake did you notice or correct during practice? - Which activity helped you understand the lesson best? - How can this skill help you design or write a Java program? - What do you still need to practice before the next lesson?

How We Show Learning

Evidence we will use to check learning: - Algorithm solves selected problem. - Sample test proves logic. - Trace matches expected output. - Completed guided-practice work with visible corrections or annotations. - A short explanation connecting the output to the lesson target.

Student Activities

Problem-to-Algorithm Challenge. Trace Table. Logic Check. Rewrite Challenge. Peer Debug. Teacher Quick Check. Create algorithms for total fare, change from payment, and class average. Write sample test data. Revise one algorithm after tracing.

Resources / References

JDK, IDE, projector or screenshots, printed worksheets, notebook, sample code snippets, and teacher-created answer guides.

Program Design Using Flowcharts And Pseudocode

Overview

We convert algorithms into flowcharts and pseudocode so program logic can be planned, traced, reviewed, and improved before Java coding.

Essential Question

How can visual and written plans make program logic easier to understand, test, and communicate?

Performance Task

Design, trace, explain, and revise both a flowchart and pseudocode solution for a simple decision-based programming problem.

Success Criteria

- Flowcharts use correct symbols, arrows, and branch labels.
- Pseudocode uses clear sequence, inputs, processes, outputs, and decisions.
- Both representations solve the same problem consistently.
- Sample values are used to trace and verify the logic.
- Revisions are explained using evidence from tracing or peer review.

LESSON 9

Lesson 9: Flowchart Symbols And Program Logic

What We'll Learn

By the end of this lesson, we can: - Identify the basic flowchart symbols and their uses. - Use the correct symbol for Start, End, Process, Input/Output, Decision, and Flowline. - Trace a simple flowchart from start to end. - Explain the logic shown by a flowchart using our own words.

Words We'll Use

- Flowchart - a visual diagram that shows the steps of a process or program.
- Terminal - oval symbol for Start or End.
- Process - a rectangle used for actions, calculations, or assignments.
- Input/Output - a parallelogram used for entering data or showing results.
- Decision - a diamond used for questions with Yes/No or True/False answers.
- Flowline - an arrow that shows the direction of the logic.
- Program Logic - the order and reasoning behind how a program works.

Before We Start

Why might a drawing help us understand a program before writing the code?

Possible student ideas: - A drawing makes the steps easier to see. - It helps us plan before coding. - It helps us find missing steps. - It helps us explain the program to others.

Let's Understand

Flowchart symbol guide A flowchart is a visual plan of a program or process. Instead of writing code immediately, we first draw the logic using standard symbols.

Each symbol has a specific purpose: 1. Terminal / Oval - Used for the beginning and ending of a flowchart. Example: Start, End 2. Process / Rectangle - Used for actions, calculations, or assignments. Example: sum = num1 + num2 3. Input/Output / Parallelogram - Used when the program receives input or displays output. Example: Input name, Display total 4. Decision / Diamond - Used when the program asks a question with two possible answers. Example: grade >= 75? 5. Flowline / Arrow Used to show which step happens next.

Important rules: - A flowchart should begin with Start. - A flowchart should end with End. - Arrows should clearly show the direction of the logic. - Use the correct symbol for each step. - A decision symbol should have labeled branches such as Yes/No or True/False. - The flowchart should be easy to trace from beginning to end.

Let's Look at Examples

Example 1: Sequence - Display a Welcome Message

A simple program displays a welcome message to the user. This example shows a straight sequence from Start to End.

> **Common mistake**
> Do not use a diamond here because the program is not asking a Yes/No question.

> **Mini-check**
> Which shape is used for displaying the message?

Example 2: Input and Output - Greet the User

The program asks for the user's name, then displays a greeting using that name.

> **Common mistake**
> Do not place 'Input name' inside a rectangle. Input and output steps should use a parallelogram.

> **Mini-check**
> Why is 'Input name' written inside a parallelogram?

Example 3: Process - Add Two Numbers

The program receives two numbers, computes their sum, and displays the result.

> **Common mistake**
> Do not use a parallelogram for 'sum = num1 + num2' because it is not receiving or displaying data. It is calculating a value.

> **Mini-check**
> Which step creates a new value?

Example 4: Decision - Check if a Student Passed

The program receives a grade. If the grade is 75 or higher, it displays Passed. Otherwise, it displays Try Again.

> **Common mistake**
> Do not forget to label the two arrows from the diamond. Without Yes and No labels, the reader may not know which path to follow.

> **Mini-check**
> What are the two possible answers to the decision 'grade >= 75?'

Example 5: Complete Logic - Compute Average and Check Result

The program receives quiz, exam, and activity scores. It computes the average, then checks if the average is passing.

> **Common mistake**
> Do not check 'average >= 75?' before computing the average. The value must be created first before it can be tested.

> **Mini-check**
> Why should the average be computed before the decision diamond?

Let's Practice

Practice 1: Identify the Symbol

Write the correct flowchart symbol for each item.

1. Start
2. Input age
3. total = price * quantity
4. Display "Welcome"
5. age >= 18?
6. End
7. Arrow showing the next step

Practice 2: Match the Symbol to Its Use

Match Column A with Column B.

Column A: 1. Oval 2. Rectangle 3. Parallelogram 4. Diamond 5. Arrow

Column B: A. Shows direction B. Used for decision C. Used for Start and End D. Used for input and output E. Used for process or calculation

Now We Apply

Create a simple flowchart for this problem:

A program asks for a student’s name and grade, then displays the name and grade.

Required steps:

1. Start
2. Input student name
3. Input grade
4. Display student name
5. Display grade
6. End

Students should draw the flowchart using the correct symbols.

Challenge Task

Create a flowchart for this problem:

A program asks for the price of an item and the quantity bought. It computes the total amount and displays the total.

Challenge question: Which step uses a rectangle? Why?

How Our Work Will Be Checked

Your work will be checked using these criteria:

Criteria	Excellent	Good	Needs Improvement
Correct Symbols	All symbols are used correctly	Most symbols are correct	Many symbols are incorrect
Complete Flow	Has Start, steps, and End	Missing one minor part	Missing important parts
Arrows and Direction	Arrows are clear and connected	Some arrows need improvement	Arrows are confusing or missing
Logic	Steps are in the correct order	Few steps are slightly unclear	Logic is hard to follow
Explanation	Can explain the flow clearly	Explanation is partly clear	Cannot explain the flow

Let's Reflect

Answer these questions: 1. Why is it important to use the correct flowchart symbol? 2. Which symbol is easiest for you to remember? 3. Which symbol is most confusing for you? 4. How can flowcharts help us before coding? 5. What mistake should you avoid when making a flowchart?

How We Show Learning

tudents will show learning by: - Identifying flowchart symbols correctly. - Drawing a simple flowchart. - Tracing a flowchart from Start to End. - Explaining what each symbol means. - Explaining the logic of their flowchart.

Student Activities

Symbol Sorting Drill. Wrong Shape Correction. Mini Flow Trace. Symbol Choice Explanation. Shape Memory Exit Ticket. Match each symbol to its purpose. Correct wrong-symbol examples. Explain the shape choice for five program steps.

Resources / References

JDK, IDE, projector or screenshots, printed worksheets, notebook, sample code snippets, and teacher-created answer guides.

LESSON 10

Lesson 10: Sequence Flowcharts For Program Design

What We'll Learn

By the end of this lesson, we can: 1. Explain what a sequence flowchart is. 2. Arrange program steps in the correct order. 3. Create a sequence flowchart using input, process, and output. 4. Trace a sequence flowchart from Start to End. 5. Convert a simple problem into a step-by-step flowchart.

Words We'll Use

Sequence - steps that happen one after another. Input - data entered by the user. Process - an action or calculation done by the program. Output - the result shown by the program. IPO - Input, Process, Output. Trace - to follow the steps of a flowchart. Algorithm - a step-by-step plan for solving a problem. Program Design - planning how a program will work before coding.

Before We Start

What happens if we write program steps in the wrong order?

Example:

If we display the total before computing it, the program will not show the correct answer.

Let's Understand

Flowchart sequence and decision guide A sequence flowchart shows steps that happen in order from top to bottom. There is no branching yet. The program simply follows one path from Start to End.

A sequence usually follows the IPO pattern: 1. Input - Get the needed data. 2. Process - Compute or perform an action. 3. Output - Show the result.

Example:

Problem: Add two numbers.

Input: - num1 - num2 Process: - sum = num1 + num2 Output: - Display sum Flow: 1. Start → Input num1 → Input num2 → sum = num1 + num2 → Display sum → End

Important rules for sequence flowcharts: - Steps must be arranged in the correct order. - Use a parallelogram for input and output. - Use a rectangle for calculations or assignments. - Use arrows to connect each step. - The flow should be easy to follow from Start to End. - There should be no decision diamond in a pure sequence flowchart.

Let's Look at Examples

Example 1

Example 2

Example 3

Let's Practice

Practice 1: Arrange the Steps

Arrange the steps in the correct order.

Problem: Compute the total price.

Steps: - Display total - Input price - End - total = price * quantity - Start - Input quantity

---

Correct order: 1. Start 2. Input price 3. Input quantity 4. total = price * quantity 5. Display total 6. End

Practice 2: Identify IPO

For each problem, identify the Input, Process, and Output. 1. Compute the sum of two numbers. 2. Compute the area of a square. 3. Compute the average of four grades. 4. Convert minutes to seconds. 5. Compute the change from payment and price.

Practice 3: Trace the Flow

Trace this flowchart: Start → Input price → Input quantity → total = price * quantity → Display total → End

---

Answer: 1. The program starts. 2. The user enters the price. 3. The user enters the quantity. 4. The program multiplies price and quantity. 5. The program displays the total. 6. The program ends.

Now We Apply

Create a sequence flowchart for this problem:

A program asks for the user’s first name and last name, combines them into a full name, then displays the full name.

Required logic: 1. Start 2. Input firstName 3. Input lastName 4. fullName = firstName + " " + lastName 5. Display fullName 6. End

Students should use: - Oval for Start and End - Parallelogram for Input and Output - Rectangle for the process - Arrows to show the correct order

Challenge Task

Create a sequence flowchart for this problem: A sari-sari store program asks for the price of one item, quantity bought, and payment. It computes the total and change, then displays both.

Challenge questions: 1. What are the inputs? 2. What are the processes? 3. What are the outputs? 4. Why should total be computed before change?

How Our Work Will Be Checked

Criteria	Excellent	Good	Needs Improvement
Correct Sequence	All steps are in the correct order	One or two steps are misplaced	Many steps are in the wrong order
Correct Symbols	All symbols are correct	Most symbols are correct	Many symbols are incorrect
IPO Identification	Input, process, and output are clear	Some parts are unclear	IPO parts are missing
Flowlines	Arrows are clear and connected	Some arrows need improvement	Arrows are missing or confusing
Explanation	Student can explain the flow clearly	Explanation is partly clear	Student cannot explain the flow

Let's Reflect

Answer these questions: 1. Why is sequence important in programming? 2. What usually comes first in a sequence flowchart? 3. Why should output come after the process? 4. What mistake did you correct while making your flowchart? 5. How can a sequence flowchart help you write code later?

How We Show Learning

Students will show learning by: - Arranging program steps in the correct order. - Identifying input, process, and output. - Creating a sequence flowchart. - Tracing the logic from Start to End. - Explaining why the order of steps is correct.

Student Activities

Algorithm To Diagram. Arrow Direction Check. IPO Labeling. Total Price Flowchart. Sequence Trace Exit Ticket. Convert a sum algorithm into a flowchart. Create a total-price sequence flowchart. Trace the diagram with sample values.

Resources / References

JDK, IDE, projector or screenshots, printed worksheets, notebook, sample code snippets, and teacher-created answer guides.

LESSON 11

Lesson 11: Decision Flowcharts And Branching Logic

What We'll Learn

By the end of this lesson, we can: - Explain what a decision flowchart is. - Use a decision symbol for Yes/No or True/False conditions. - Label decision branches correctly. - Trace a flowchart with branching logic. - Create a decision flowchart for a simple programming problem. - Draw Yes and No branches from a decision. - Trace both branches using different sample values.

Words We'll Use

- Decision - a question or condition that leads to different paths.
- Condition - a statement that can be true or false.
- Branch - one path taken after a decision.
- Yes/No Path - the two possible directions from a decision symbol.
- True/False Path - another way to label decision branches.
- Comparison Operator - a symbol used to compare values, such as >, <, >=, <=, ==, or !=.
- Branching Logic - program logic where different actions happen depending on a condition.
- Trace - to follow the flow of the program step by step

Before We Start

Have you ever made a decision where your next action depended on the answer?

Let's Understand

Flowchart sequence and decision guide A decision flowchart is used when a program must choose between two or more possible actions.

The decision symbol is a diamond. Inside the diamond, we write a condition or question. Example:

grade >= 75?

This question has two possible answers:

- Yes / True
- No / False

If the answer is Yes, the program follows the Yes branch. If the answer is No, the program follows the No branch.

Example logic:

If grade is greater than or equal to 75, display "Passed". Otherwise, display "Failed".

Flow:

Start → Input grade → grade >= 75? Yes → Display "Passed" → End No → Display "Failed" → End

Important rules for decision flowcharts:

- Use a diamond for a condition.
- The condition should be written as a question.
- Decision branches must be labeled.
- Each branch should connect to the next correct step.
- The flowchart should still have Start and End.
- The logic should be traceable for both possible answers.
- Avoid confusing or crossing arrows when possible.

Common comparison examples:

- age >= 18?
- grade >= 75?
- password == correctPassword?
- number > 0?
- total >= 1000?

Let's Look at Examples

Example 1

Example 2

Let's Practice

Practice 1: Identify the Condition

Write the condition for each problem.

1. Check if a grade is passing.
2. Check if age is 18 or above.
3. Check if a number is greater than 0.
4. Check if total is at least 1000.
5. Check if password is correct.

Practice 2: Label the Branches

For each decision, write the Yes and No result.

1. grade >= 75? Yes: _ ***No:*** _
2. age >= 18? Yes: _ ***No:*** _
3. number > 0? Yes: _ ***No:*** _

Now We Apply

Create a decision flowchart for this problem:

A program asks for a student’s score. If the score is 50 or higher, display "Complete". Otherwise, display "Incomplete".

Required logic:

1. Start
2. Input score
3. score >= 50?
4. If Yes, display "Complete"
5. If No, display "Incomplete"
6. End

Students should use:

- Oval for Start and End
- Parallelogram for Input and Output
- Diamond for the decision
- Labeled arrows for Yes and No

Challenge Task

Create a decision flowchart for this problem:

A store gives a discount if the total purchase is 1000 pesos or more. Ask for the total purchase. If the total is 1000 or more, display "With Discount". Otherwise, display "No Discount".

Challenge extension:

Add a process step:

If totalPurchase is 1000 or more, compute:

discount = totalPurchase * 0.10

Then display the discount.

How Our Work Will Be Checked

Your work will be checked using these criteria: | Criteria | Excellent | Good | Needs Improvement | |---|---|---|---| | Correct Decision Symbol | Diamond is used correctly for the condition | Decision is present but unclear | Decision symbol is missing or wrong | | Branch Labels | Yes/No or True/False labels are clear | Some labels are unclear | Branch labels are missing | | Correct Logic | Both paths lead to correct results | One path has a minor issue | Logic is incorrect or incomplete | | Correct Symbols | All symbols are used correctly | Most symbols are correct | Many symbols are incorrect | | Traceability | Flow can be traced for both answers | One path is slightly confusing | Flow is hard to trace | | Explanation | Student explains both branches clearly | Explanation is partly clear | Student cannot explain the branches |

Let's Reflect

1. Why do we use a diamond for a decision?
2. Why should decision branches be labeled?
3. What happens if the Yes and No paths are missing?
4. How is a decision flowchart different from a sequence flowchart?
5. What real-life decision can you turn into a flowchart?

How We Show Learning

Students will show learning by:

- Writing correct conditions.
- Using decision symbols properly.
- Labeling Yes/No or True/False branches.
- Tracing both branches of a flowchart.
- Creating a decision flowchart.
- Explaining how the program chooses a path.

Student Activities

Decision Question Builder. Pass/Fail Flowchart. Trace Two Branches. Missing Branch Label Fix. Branch Logic Exit Ticket. Write decision questions. Draw a pass/fail branch flowchart. Trace one passing and one failing value.

Resources / References

JDK, IDE, projector or screenshots, printed worksheets, notebook, sample code snippets, and teacher-created answer guides.

LESSON 12

Lesson 12: Flowcharting Simple Programming Problems

What We'll Learn

1. Analyze a simple programming problem before making a flowchart.
2. Identify the needed input, process, decision, and output.
3. Create a complete flowchart for a simple programming problem.
4. Trace the flowchart using sample values.
5. Explain how the flowchart solves the problem.

Words We'll Use

Problem Analysis - understanding what the problem is asking before solving it. Input - the data needed by the program. Process - the calculation or action done by the program. Decision - a condition that chooses between two paths. Output - the result shown by the program. IPO - Input, Process, Output. Condition - a statement that can be answered as Yes/No or True/False. Trace - to follow the flowchart step by step using sample values. Algorithm - a step-by-step plan for solving a problem. Flowchart - a visual diagram that shows the logic of a program.

Before We Start

Before drawing a flowchart, why should we first understand the problem?

Let's Understand

Flowchart sequence and decision guide In this lesson, we will combine what we learned from the previous lessons. From Lesson 9, we learned the flowchart symbols:

- Oval for Start and End
- Parallelogram for Input and Output
- Rectangle for Process
- Diamond for Decision
- Arrow for Flowline

From Lesson 10, we learned sequence flowcharts:

- Steps happen one after another.
- The program follows one path from Start to End.
- Most sequence problems follow Input → Process → Output.

From Lesson 11, we learned decision flowcharts:

- The program checks a condition.
- The decision has Yes/No or True/False branches.
- Each branch should lead to the correct result.

Now, we will use all of these skills to solve simple programming problems.

How to Flowchart a Simple Programming Problem Use this step-by-step guide:

- Read the problem carefully.
- What is the program asking you to do?
- Find the inputs.
- What data should the user enter?
- Find the process.
- What calculation or action should the program perform?
- Check if there is a decision.
- Does the program need to choose between two results?
- Find the output.
- What should the program display?
- Arrange the steps in order.
- Start with Start and end with End.
- Use the correct symbols.
- Match each step with the correct flowchart shape.
- Trace the flowchart.
- Test it using sample values.

Let's Look at Examples

Example 1

Example 2

Example 3

Let's Practice

Practice 1: Identify the Parts

For each problem, identify the Input, Process, Decision, and Output.

Problem A

Ask for the length and width of a rectangle. Compute and display the area.

- Input: _____
- Process: _____
- Decision: _____
- Output: _____ Problem B

Ask for the age of a person. If the age is 18 or above, display "Adult". Otherwise, display "Minor".

- Input: _____
- Process: _____
- Decision: _____
- Output: _____ Problem C

Ask for the price and quantity of an item. Compute and display the total.

- Input: _____
- Process: _____
- Decision: _____
- Output: _____

Practice 2: Arrange the Flowchart Steps

Arrange the steps in the correct order.

Problem:

Ask for a student’s score. If the score is 50 or higher, display "Complete". Otherwise, display "Incomplete".

Steps:

- Display "Complete"
- Input score
- End
- Start
- score >= 50?
- Display "Incomplete"

Correct order:

1. Start
2. Input score
3. score >= 50?

4. Yes → Display "Complete"
5. No → Display "Incomplete"
6. End

Practice 3: Trace the Flowchart

Trace this flowchart using the given values.

Flow:

Start → Input grade → grade >= 75? Yes → Display "Passed" → End No → Display "Failed" → End

Test values:

1. grade = 80
2. grade = 70
3. grade = 75

Questions:

1. Which branch is followed?
2. What output is displayed?
3. Why?

Now We Apply

Create a complete flowchart for this problem:

A program asks for two quiz scores. It computes the average. If the average is 75 or higher, display "Passed". Otherwise, display "Failed".

Required problem analysis:

- Input: quiz1, quiz2
- Process: average = (quiz1 + quiz2) / 2
- Decision: average >= 75?
- Output: Display "Passed" or Display "Failed"

Required logic:

1. Start
2. Input quiz1
3. Input quiz2
4. average = (quiz1 + quiz2) / 2
5. average >= 75?
6. Yes → Display "Passed"
7. No → Display "Failed"
8. End

Students should use:

- Oval for Start and End
- Parallelogram for Input and Output
- Rectangle for the average computation
- Diamond for the condition
- Labeled arrows for Yes and No

Challenge Task

Create a complete flowchart for this problem:

A store program asks for the price of an item, quantity bought, and payment. It computes the total amount. If the payment is enough, display the total and change. If the payment is not enough, display "Insufficient Payment".

Challenge questions:

1. What are the inputs?
2. What is the first process?
3. What condition should be placed inside the diamond?
4. Why should total be computed before checking payment?
5. What happens if payment is less than total?

How Our Work Will Be Checked

Criteria	Excellent	Good	Needs Improvement
Problem Analysis	Input, process, decision, and output are complete and correct	Most parts are correct	Many parts are missing or incorrect
Correct Symbols	All flowchart symbols are used correctly	Most symbols are correct	Many symbols are incorrect
Correct Logic	The flow solves the problem correctly	The flow has minor logic issues	The flow does not solve the problem
Branch Labels	Yes/No branches are clearly labeled when needed	Some labels are unclear	Branch labels are missing
Sequence and Order	Steps are arranged in the correct order	Some steps are slightly misplaced	Steps are confusing or incomplete
Traceability	Flowchart can be traced using sample values	One path is slightly confusing	Flowchart is hard to trace
Explanation	Student explains the solution clearly	Explanation is partly clear	Student cannot explain the solution

Let's Reflect

1. What should you do first before drawing a flowchart?
2. How do you know if a problem needs a decision symbol?
3. Why is it important to identify the input, process, and output?
4. What part of creating a flowchart is easiest for you?
5. What part is still difficult for you?
6. How can flowcharting help you when you start coding?

How We Show Learning

Students will show learning by:

Analyzing a simple programming problem. - Identifying input, process, decision, and output. - Creating a complete flowchart. - Using correct flowchart symbols. - Labeling branches correctly. - Tracing the flowchart using sample values. - Explaining how their flowchart solves the problem.

Student Activities

Problem Analysis. Full Flowchart Build. Sample Grade Trace. Peer Review Checklist. Revision Reflection. Plan the IPO for average grade. Draw a complete flowchart with sequence and decision logic. Revise after tracing two sample grade sets.

Resources / References

JDK, IDE, projector or screenshots, printed worksheets, notebook, sample code snippets, and teacher-created answer guides.

LESSON 13

Lesson 13: Pseudocode Fundamentals For Program Planning

What We'll Learn

- Explain what pseudocode is.
- Describe why pseudocode is useful before coding.
- Write simple step-by-step instructions using plain language.
- Arrange pseudocode steps in the correct order.
- Compare pseudocode with flowcharts and actual code.

Words We'll Use

- Pseudocode - a way of planning a program using simple English-like instructions.
- Algorithm - a step-by-step plan for solving a problem.
- Instruction - one action the program should perform.
- Sequence - steps that happen one after another.
- Program Planning - preparing the logic of a program before writing actual code.
- Plain Language - simple words that are easy to understand.
- Logic - the order and reasoning behind how a program works.

Before We Start

Before writing code, why is it helpful to write the steps in simple words first?

Let's Understand

Pseudocode pattern guide Pseudocode is a simple way to plan a program before writing actual code. It is not a real programming language, so we do not need to worry too much about syntax.

Instead of using Java syntax immediately, we first write the logic in simple English-like steps.

Example:

Problem: Display a welcome message. Pseudocode:

```
Pseudocode
Start
 Display "Welcome"
End
```

Pseudocode helps us because:

- It makes the program easier to understand.
- It helps us organize our ideas.
- It helps us find missing steps.
- It helps us prepare before coding.
- It can be converted into a flowchart or actual code later.

Pseudocode vs Flowchart vs Code

Type	Meaning	Example
Flowchart	Visual diagram of logic	Start → Display "Hello" → End
Pseudocode	Plain-language steps	Display "Hello"
Code	Actual programming language	`System.out.println("Hello");`

Basic Pseudocode Rules

1. Start with START.
2. End with END.
3. Write one instruction per line.
4. Use clear action words like Input, Display, Compute, Set, or Check.
5. Arrange the steps in the correct order.
6. Keep the language simple.
7. Do not worry too much about exact Java syntax yet.

Common Pseudocode Action Words

- INPUT - receive data from the user
- DISPLAY - show output
- SET - assign a value
- COMPUTE - calculate a result
- IF - start a decision
- THEN - result if the condition is true
- ELSE - result if the condition is false
- END IF - end a decision block

Let's Look at Examples

Example 1

Problem:

Write pseudocode that displays "Hello, Student!"

```
Pseudocode:

START
 DISPLAY "Hello, Student!"
END
```

Explanation:

The program starts, displays the message, and ends.

Example 2

Problem:

Write pseudocode that asks for a name and displays it.

```
Pseudocode:

START
INPUT name
 DISPLAY name
END
```

Explanation:

The program asks for the user’s name, then displays the same name.

Example 3

Problem:

Write pseudocode that asks for two numbers, adds them, and displays the sum.

```
Pseudocode:

START
 INPUT num1
 INPUT num2
 COMPUTE sum = num1 + num2
 DISPLAY sum
END
```

Explanation:

The program receives two numbers, computes their sum, and displays the result.

Practice 1: Identify the Purpose

Tell whether each line is input, process, or output.

1. INPUT age
2. DISPLAY "Welcome"
3. COMPUTE total = price * quantity
4. INPUT grade
5. DISPLAY average

Practice 2: Arrange the Steps

Arrange the pseudocode steps in the correct order.

Problem:

Ask for a name and display it.

Steps:

- END
- DISPLAY name
- START
- INPUT name

Practice 3: Write Simple Pseudocode

Write pseudocode for each problem.

1. Display your school name.
2. Ask for a student name and display it.
3. Ask for a favorite color and display it.
4. Ask for an age and display it.

Now We Apply

Write pseudocode for this problem:

A program asks for a student’s first name and last name, then displays the full name.

Required logic:

1. Start
2. Input firstName
3. Input lastName
4. Set fullName = firstName + " " + lastName
5. Display fullName
6. End

Expected pseudocode:

START INPUT firstName INPUT lastName SET fullName = firstName + " " + lastName DISPLAY fullName END

Challenge Task

Write pseudocode for this problem:

A program asks for the name of a product and its price, then displays both the product name and price.

Challenge questions:

1. What are the inputs?
2. Is there a process?
3. What should be displayed?
4. Why should the input happen before the output?

How Our Work Will Be Checked

Criteria	Excellent	Good	Needs Improvement
Correct Order	Steps are arranged correctly from START to END	One step is slightly misplaced	Steps are confusing or incorrect
Clear Instructions	Instructions are easy to understand	Some instructions need improvement	Instructions are unclear
Correct Use of Action Words	Uses INPUT, DISPLAY, SET, or COMPUTE correctly	Some action words are incorrect	Action words are missing or wrong
Complete Logic	All needed steps are present	One minor step is missing	Important steps are missing
Explanation	Student explains the pseudocode clearly	Explanation is partly clear	Student cannot explain the pseudocode

Let's Reflect

1. What is pseudocode?
2. Why do we use pseudocode before coding?
3. How is pseudocode different from a flowchart?
4. Which is easier for you: flowchart or pseudocode? Why?
5. What makes pseudocode clear and easy to follow?

How We Show Learning

Students will show learning by: - Explaining the purpose of pseudocode. - Writing simple pseudocode. - Arranging pseudocode steps correctly. - Using clear action words. - Explaining the logic of their pseudocode.

Student Activities

Identify Pseudocode. Algorithm to Pseudocode. Complete Missing Lines. Trace with Values. Pseudocode Fix-It Task. Correction Log. Write pseudocode for area of rectangle. Write pseudocode for grade status. Exchange and revise with a partner.

Resources / References

JDK, IDE, projector or screenshots, printed worksheets, notebook, sample code snippets, and teacher-created answer guides.

LESSON 14

Lesson 14: Pseudocode With Input, Process, And Output

What We'll Learn

By the end of this lesson, we can: - Write readable pseudocode. - Use input, process, output, and decisions when needed. - Revise pseudocode so the logic is complete.

Words We'll Use

- Pseudocode - plain-language program plan.
- INPUT - receive data.
- SET - assign or compute value.
- DISPLAY - show result.
- IF/ELSE - decision structure.

Before We Start

Why might programmers write a plan before writing Java code?

Let's Understand

Pseudocode pattern guide Let's work through Lesson 14: Pseudocode With Input, Process, And Output together. Our focus is plain-English program planning before Java coding. We will move from meaning, to examples, to practice, then to a task we can explain. What this lesson means: - Pseudocode is a plain-language plan for a program. - We use simple keywords like START, INPUT, SET, DISPLAY, IF, ELSE, and END. - Pseudocode is not Java yet, so it does not need semicolons or exact Java syntax. - It still needs clear order, complete logic, and consistent variable names. - Indentation helps us see which steps belong inside a decision. - We trace pseudocode with sample values before turning it into code. How we study it step by step: - First, we connect the lesson to a familiar situation: Why might programmers write a plan before writing Java code? - Next, we name the important parts so everyone uses the same vocabulary. - Then, we study Pseudocode keywords and ask: what does it do, where do we see it, and how can we check it? - Then, we study IPO and ask: what does it do, where do we see it, and how can we check it? - Then, we study IF/ELSE and ask: what does it do, where do we see it, and how can we check it? - Then, we study Revision and ask: what does it do, where do we see it, and how can we check it? - After that, we compare a correct example with a common wrong version. - We trace the example slowly before running, drawing, or submitting anything. - Finally, we explain the result in our own words so the activity is not just copying. Rules and patterns we should remember: - Rule: Use START and END to show the boundary of the plan. Example: `START` begins the pseudocode and `END` closes it. Check: Where should the first and last lines be? - Rule: Use INPUT for values we need from the user. Example: `INPUT grade` means the user or problem gives us a grade value. Check: What value are we receiving? - Rule: Use SET for assignment or computation. Example: `SET average = total / 3` stores the computed average. Check: What variable receives the result? - Rule: Use DISPLAY for the result. Example: `DISPLAY average` shows the computed value. Check: What should the user see? - Rule: Use IF, ELSE, and END IF for decisions. Example: `IF grade >= 75 THEN DISPLAY "Passed" ELSE DISPLAY "Needs practice" END IF` handles two paths. Check: What condition decides the path? - Rule: Keep variable names consistent from beginning to end. Example: If we use `grade` in INPUT, use `grade` again in IF, not `score` unless we define it. Check: Did the variable name change? Common mistakes we will watch for: - Writing Java syntax instead of a clear plan. - Skipping INPUT or DISPLAY steps. - Changing variable names halfway through. - Forgetting END IF after a decision. - Not testing the pseudocode with sample data. Mini-check before practice: - What keyword shows that we receive data? - What keyword shows that we display a result? - Where should the steps inside an IF decision be placed? - Can we trace the pseudocode using sample values? When we move to practice, our goal is not only to get an answer. Our goal is to explain the thinking: what information we used, what step happened next, why the result is correct, and what we changed after testing.

Let's Practice

Let's practice with these activities:

Activity 1: Pseudocode Keyword Check Type: individual Time: 7 minutes Instruction: Review the keywords we use for program planning. Student task: - START means ***_. - INPUT means*** *. - SET means* ***_. - DISPLAY means*** . - IF/ELSE means ____. Output: Submit five completed keyword meanings.

Activity 2: Guided Pseudocode Task Type: individual Time: 12 minutes Instruction: Write or revise pseudocode for this lesson focus. Student task: - Write pseudocode for sum of two numbers. - Use INPUT, SET, and DISPLAY. - Trace with num1 = 6 and num2 = 9. Output: Submit complete pseudocode and trace notes.

Activity 3: Complete The Missing Lines Type: pair Time: 10 minutes Instruction: Fill in missing pseudocode lines. Student task: - START - INPUT grade - ***_ average = grade - IF average >= 75 THEN -*** _ "Passed" - ELSE - DISPLAY "Try again" - ____ IF - END Output: Submit completed missing lines.

Activity 4: Trace With Values Type: pair Time: 8 minutes Instruction: Trace pseudocode manually before coding. Student task: - Choose two sample inputs. - Write each variable value after SET. - Write the displayed output. - Mark which IF path was followed. Output: Submit a two-row trace table.

Activity 5: Exit Ticket Type: exit ticket Time: 5 minutes Instruction: Check pseudocode readiness. Student task: - What keyword receives data? - What keyword shows output? - Why do we trace pseudocode? Output: Submit three answers.

Now We Apply

Now we apply it through these tasks: 1. Write pseudocode for area of rectangle. 2. Write pseudocode for grade status. 3. Exchange and revise with a partner.

Challenge Task

Correct broken pseudocode for computing average grade and pass/fail status, then submit a revision log.

How Our Work Will Be Checked

Criteria - 20 points total - Correctness (5): Output or answer follows the required concept and instructions. - Completeness (4): All required parts are present. - Logic and sequence (4): Steps, code, diagram, or pseudocode follow a clear order. - Explanation (3): Student can explain what the work does and why it is correct. - Neatness/readability (2): Work is easy to read, label, and check. - Effort and revision (2): Student improves the work after feedback.

Let's Reflect

Let's reflect: - What concept from this lesson is clearest to you now? - What mistake did you notice or correct during practice? - Which activity helped you understand the lesson best? - How can this skill help you design or write a Java program? - What do you still need to practice before the next lesson?

How We Show Learning

Evidence we will use to check learning: - Pseudocode is readable and ordered. - Logic is complete. - Revision log explains changes. - Completed guided-practice work with visible corrections or annotations. - A short explanation connecting the output to the lesson target.

Student Activities

Identify Pseudocode. Algorithm to Pseudocode. Complete Missing Lines. Trace with Values. Pseudocode Fix-It Task. Correction Log. Write pseudocode for area of rectangle. Write pseudocode for grade status. Exchange and revise with a partner.

Resources / References

JDK, IDE, projector or screenshots, printed worksheets, notebook, sample code snippets, and teacher-created answer guides.

LESSON 15

Lesson 15: Pseudocode With Decisions And Branches

What We'll Learn

By the end of this lesson, we can: - Write readable pseudocode. - Use input, process, output, and decisions when needed. - Revise pseudocode so the logic is complete.

Words We'll Use

- Pseudocode - plain-language program plan.
- INPUT - receive data.
- SET - assign or compute value.
- DISPLAY - show result.
- IF/ELSE - decision structure.

Before We Start

Why might programmers write a plan before writing Java code?

Let's Understand

Pseudocode pattern guide Let's work through Lesson 15: Pseudocode With Decisions And Branches together. Our focus is plain-English program planning before Java coding. We will move from meaning, to examples, to practice, then to a task we can explain. What this lesson means: - Pseudocode is a plain-language plan for a program. - We use simple keywords like START, INPUT, SET, DISPLAY, IF, ELSE, and END. - Pseudocode is not Java yet, so it does not need semicolons or exact Java syntax. - It still needs clear order, complete logic, and consistent variable names. - Indentation helps us see which steps belong inside a decision. - We trace pseudocode with sample values before turning it into code. How we study it step by step: - First, we connect the lesson to a familiar situation: Why might programmers write a plan before writing Java code? - Next, we name the important parts so everyone uses the same vocabulary. - Then, we study Pseudocode keywords and ask: what does it do, where do we see it, and how can we check it? - Then, we study IPO and ask: what does it do, where do we see it, and how can we check it? - Then, we study IF/ELSE and ask: what does it do, where do we see it, and how can we check it? - Then, we study Revision and ask: what does it do, where do we see it, and how can we check it? - After that, we compare a correct example with a common wrong version. - We trace the example slowly before running, drawing, or submitting anything. - Finally, we explain the result in our own words so the activity is not just copying. Rules and patterns we should remember: - Rule: Use START and END to show the boundary of the plan. Example: `START` begins the pseudocode and `END` closes it. Check: Where should the first and last lines be? - Rule: Use INPUT for values we need from the user. Example: `INPUT grade` means the user or problem gives us a grade value. Check: What value are we receiving? - Rule: Use SET for assignment or computation. Example: `SET average = total / 3` stores the computed average. Check: What variable receives the result? - Rule: Use DISPLAY for the result. Example: `DISPLAY average` shows the computed value. Check: What should the user see? - Rule: Use IF, ELSE, and END IF for decisions. Example: `IF grade >= 75 THEN DISPLAY "Passed" ELSE DISPLAY "Needs practice" END IF` handles two paths. Check: What condition decides the path? - Rule: Keep variable names consistent from beginning to end. Example: If we use `grade` in INPUT, use `grade` again in IF, not `score` unless we define it. Check: Did the variable name change? Common mistakes we will watch for: - Writing Java syntax instead of a clear plan. - Skipping INPUT or DISPLAY steps. - Changing variable names halfway through. - Forgetting END IF after a decision. - Not testing the pseudocode with sample data. Mini-check before practice: - What keyword shows that we receive data? - What keyword shows that we display a result? - Where should the steps inside an IF decision be placed? - Can we trace the pseudocode using sample values? When we move to practice, our goal is not only to get an answer. Our goal is to explain the thinking: what information we used, what step happened next, why the result is correct, and what we changed after testing.

Let's Practice

Let's practice with these activities:

Activity 1: Pseudocode Keyword Check Type: individual Time: 7 minutes Instruction: Review the keywords we use for program planning. Student task: - START means ***_. - INPUT means*** *. - SET means* ***_. - DISPLAY means*** . - IF/ELSE means ____. Output: Submit five completed keyword meanings.

Activity 2: Guided Pseudocode Task Type: individual Time: 12 minutes Instruction: Write or revise pseudocode for this lesson focus. Student task: - Write pseudocode for pass/fail grade. - Use IF, ELSE, and END IF. - Trace grade = 80 and grade = 70. Output: Submit complete pseudocode and trace notes.

Activity 3: Complete The Missing Lines Type: pair Time: 10 minutes Instruction: Fill in missing pseudocode lines. Student task: - START - INPUT grade - ***_ average = grade - IF average >= 75 THEN -*** _ "Passed" - ELSE - DISPLAY "Try again" - ____ IF - END Output: Submit completed missing lines.

Activity 4: Trace With Values Type: pair Time: 8 minutes Instruction: Trace pseudocode manually before coding. Student task: - Choose two sample inputs. - Write each variable value after SET. - Write the displayed output. - Mark which IF path was followed. Output: Submit a two-row trace table.

Activity 5: Exit Ticket Type: exit ticket Time: 5 minutes Instruction: Check pseudocode readiness. Student task: - What keyword receives data? - What keyword shows output? - Why do we trace pseudocode? Output: Submit three answers.

Now We Apply

Now we apply it through these tasks: 1. Write pseudocode for area of rectangle. 2. Write pseudocode for grade status. 3. Exchange and revise with a partner.

Challenge Task

Correct broken pseudocode for computing average grade and pass/fail status, then submit a revision log.

How Our Work Will Be Checked

Criteria - 20 points total - Correctness (5): Output or answer follows the required concept and instructions. - Completeness (4): All required parts are present. - Logic and sequence (4): Steps, code, diagram, or pseudocode follow a clear order. - Explanation (3): Student can explain what the work does and why it is correct. - Neatness/readability (2): Work is easy to read, label, and check. - Effort and revision (2): Student improves the work after feedback.

Let's Reflect

Let's reflect: - What concept from this lesson is clearest to you now? - What mistake did you notice or correct during practice? - Which activity helped you understand the lesson best? - How can this skill help you design or write a Java program? - What do you still need to practice before the next lesson?

How We Show Learning

Evidence we will use to check learning: - Pseudocode is readable and ordered. - Logic is complete. - Revision log explains changes. - Completed guided-practice work with visible corrections or annotations. - A short explanation connecting the output to the lesson target.

Student Activities

Identify Pseudocode. Algorithm to Pseudocode. Complete Missing Lines. Trace with Values. Pseudocode Fix-It Task. Correction Log. Write pseudocode for area of rectangle. Write pseudocode for grade status. Exchange and revise with a partner.

Resources / References

JDK, IDE, projector or screenshots, printed worksheets, notebook, sample code snippets, and teacher-created answer guides.

LESSON 16

Lesson 16: Pseudocode Review, Fix-It, And Revision

What We'll Learn

By the end of this lesson, we can: - Write readable pseudocode. - Use input, process, output, and decisions when needed. - Revise pseudocode so the logic is complete.

Words We'll Use

- Pseudocode - plain-language program plan.
- INPUT - receive data.
- SET - assign or compute value.
- DISPLAY - show result.
- IF/ELSE - decision structure.

Before We Start

Why might programmers write a plan before writing Java code?

Let's Understand

Pseudocode pattern guide Let's work through Lesson 16: Pseudocode Review, Fix-It, And Revision together. Our focus is plain-English program planning before Java coding. We will move from meaning, to examples, to practice, then to a task we can explain. What this lesson means: - Pseudocode is a plain-language plan for a program. - We use simple keywords like START, INPUT, SET, DISPLAY, IF, ELSE, and END. - Pseudocode is not Java yet, so it does not need semicolons or exact Java syntax. - It still needs clear order, complete logic, and consistent variable names. - Indentation helps us see which steps belong inside a decision. - We trace pseudocode with sample values before turning it into code. How we study it step by step: - First, we connect the lesson to a familiar situation: Why might programmers write a plan before writing Java code? - Next, we name the important parts so everyone uses the same vocabulary. - Then, we study Pseudocode keywords and ask: what does it do, where do we see it, and how can we check it? - Then, we study IPO and ask: what does it do, where do we see it, and how can we check it? - Then, we study IF/ELSE and ask: what does it do, where do we see it, and how can we check it? - Then, we study Revision and ask: what does it do, where do we see it, and how can we check it? - After that, we compare a correct example with a common wrong version. - We trace the example slowly before running, drawing, or submitting anything. - Finally, we explain the result in our own words so the activity is not just copying. Rules and patterns we should remember: - Rule: Use START and END to show the boundary of the plan. Example: `START` begins the pseudocode and `END` closes it. Check: Where should the first and last lines be? - Rule: Use INPUT for values we need from the user. Example: `INPUT grade` means the user or problem gives us a grade value. Check: What value are we receiving? - Rule: Use SET for assignment or computation. Example: `SET average = total / 3` stores the computed average. Check: What variable receives the result? - Rule: Use DISPLAY for the result. Example: `DISPLAY average` shows the computed value. Check: What should the user see? - Rule: Use IF, ELSE, and END IF for decisions. Example: `IF grade >= 75 THEN DISPLAY "Passed" ELSE DISPLAY "Needs practice" END IF` handles two paths. Check: What condition decides the path? - Rule: Keep variable names consistent from beginning to end. Example: If we use `grade` in INPUT, use `grade` again in IF, not `score` unless we define it. Check: Did the variable name change? Common mistakes we will watch for: - Writing Java syntax instead of a clear plan. - Skipping INPUT or DISPLAY steps. - Changing variable names halfway through. - Forgetting END IF after a decision. - Not testing the pseudocode with sample data. Mini-check before practice: - What keyword shows that we receive data? - What keyword shows that we display a result? - Where should the steps inside an IF decision be placed? - Can we trace the pseudocode using sample values? When we move to practice, our goal is not only to get an answer. Our goal is to explain the thinking: what information we used, what step happened next, why the result is correct, and what we changed after testing.

Let's Practice

Let's practice with these activities:

Activity 1: Pseudocode Keyword Check Type: individual Time: 7 minutes Instruction: Review the keywords we use for program planning. Student task: - START means ***_. - INPUT means*** *. - SET means* ***_. - DISPLAY means*** . - IF/ELSE means ____. Output: Submit five completed keyword meanings.

Activity 2: Guided Pseudocode Task Type: individual Time: 12 minutes Instruction: Write or revise pseudocode for this lesson focus. Student task: - Find missing INPUT, SET, DISPLAY, or END IF lines. - Rewrite broken pseudocode. - Explain each correction. Output: Submit complete pseudocode and trace notes.

Activity 3: Complete The Missing Lines Type: pair Time: 10 minutes Instruction: Fill in missing pseudocode lines. Student task: - START - INPUT grade - ***_ average = grade - IF average >= 75 THEN -*** _ "Passed" - ELSE - DISPLAY "Try again" - ____ IF - END Output: Submit completed missing lines.

Activity 4: Trace With Values Type: pair Time: 8 minutes Instruction: Trace pseudocode manually before coding. Student task: - Choose two sample inputs. - Write each variable value after SET. - Write the displayed output. - Mark which IF path was followed. Output: Submit a two-row trace table.

Activity 5: Exit Ticket Type: exit ticket Time: 5 minutes Instruction: Check pseudocode readiness. Student task: - What keyword receives data? - What keyword shows output? - Why do we trace pseudocode? Output: Submit three answers.

Now We Apply

Now we apply it through these tasks: 1. Write pseudocode for area of rectangle. 2. Write pseudocode for grade status. 3. Exchange and revise with a partner.

Challenge Task

Correct broken pseudocode for computing average grade and pass/fail status, then submit a revision log.

How Our Work Will Be Checked

Criteria - 20 points total - Correctness (5): Output or answer follows the required concept and instructions. - Completeness (4): All required parts are present. - Logic and sequence (4): Steps, code, diagram, or pseudocode follow a clear order. - Explanation (3): Student can explain what the work does and why it is correct. - Neatness/readability (2): Work is easy to read, label, and check. - Effort and revision (2): Student improves the work after feedback.

Let's Reflect

Let's reflect: - What concept from this lesson is clearest to you now? - What mistake did you notice or correct during practice? - Which activity helped you understand the lesson best? - How can this skill help you design or write a Java program? - What do you still need to practice before the next lesson?

How We Show Learning

Evidence we will use to check learning: - Pseudocode is readable and ordered. - Logic is complete. - Revision log explains changes. - Completed guided-practice work with visible corrections or annotations. - A short explanation connecting the output to the lesson target.

Student Activities

Identify Pseudocode. Algorithm to Pseudocode. Complete Missing Lines. Trace with Values. Pseudocode Fix-It Task. Correction Log. Write pseudocode for area of rectangle. Write pseudocode for grade status. Exchange and revise with a partner.

Resources / References

JDK, IDE, projector or screenshots, printed worksheets, notebook, sample code snippets, and teacher-created answer guides.

Java Operators, Input, Output, And File Handling

Overview

We use Java variables, data types, operators, expressions, user input, clean output, and basic file reading/writing to build beginner Java programs.

Essential Question

How can Java receive, process, display, and preserve information accurately and responsibly?

Performance Task

Create and explain a Student Information File program that reads user input, processes values, displays a clear summary, writes data to a file, and verifies the saved result.

Success Criteria

- Variables and data types match the information being stored.
- Operators and expressions produce accurate results.
- Prompts and displayed output are clear and labeled.
- File writing and reading follow the required Java I/O steps.
- Testing evidence proves the console and file results are correct.

LESSON 17

Lesson 17: Java Variables And Data Types

What We'll Learn

By the end of this lesson, we can: - Choose suitable data types. - Use arithmetic, assignment, comparison, or logical operators. - Predict and explain expression results.

Words We'll Use

- Variable - named storage.
- Data type - kind of value.
- Operator - symbol that performs an action.
- Expression - code that produces a value.
- Boolean expression - true/false result.

Before We Start

What information about a student can a program store, and what operation might the program perform on it?

Let's Understand

Java operators reference Let's work through Lesson 17: Java Variables And Data Types together. Our focus is using variables, data types, and operators to build Java expressions. We will move from meaning, to examples, to practice, then to a task we can explain. What this lesson means: - Variables store values so a program can use them later. - Data types tell Java what kind of value we are storing, such as int, double, char, boolean, or String. - Operators are symbols that perform actions on values. - Arithmetic operators compute numbers. Comparison operators produce true or false. - Logical operators combine conditions so programs can make better decisions. - We trace expressions carefully because order and parentheses can change the result. How we study it step by step: - First, we connect the lesson to a familiar situation: What information about a student can a program store, and what operation might the program perform on it? - Next, we name the important parts so everyone uses the same vocabulary. - Then, we study Variables and ask: what does it do, where do we see it, and how can we check it? - Then, we study Data types and ask: what does it do, where do we see it, and how can we check it? - Then, we study Operators and ask: what does it do, where do we see it, and how can we check it? - Then, we study Expressions and ask: what does it do, where do we see it, and how can we check it? - After that, we compare a correct example with a common wrong version. - We trace the example slowly before running, drawing, or submitting anything. - Finally, we explain the result in our own words so the activity is not just copying. Rules and patterns we should remember: - Rule: Use = to assign a value and == to compare equality. Example: `score = 90;` stores 90, while `score == 90` asks if score is equal to 90. Check: Are we storing a value or asking a true/false question? - Rule: Use +, -, *, /, and % for arithmetic. Example: `int remainder = 10 % 3;` stores 1 because 10 divided by 3 leaves 1. Check: What does % give us? - Rule: Use parentheses when we want a calculation to happen first. Example: `average = (g1 + g2 + g3) / 3.0;` adds first, then divides. Check: What happens first inside the parentheses? - Rule: Use && when both conditions must be true. Example: `grade >= 75 && attendance >= 80` is true only when both checks pass. Check: What if attendance is only 70? - Rule: Use || when at least one condition may be true. Example: `hasPermit || isTeacherApproved` is true if either side is true. Check: How many sides need to be true? - Rule: Use ! to reverse a boolean condition. Example: `!isAbsent` means the student is not absent. Check: If `isAbsent` is false, what is `!isAbsent`? Common mistakes we will watch for: - Using String for values that should be numeric. - Confusing assignment = with comparison ==. - Forgetting parentheses in average formulas. - Expecting integer division to keep decimals. - Reading && and || without checking both sides of the condition. Mini-check before practice: - What data type fits this value? - Are we assigning a value or comparing two values? - Which operation happens first? - Does the final expression produce a number or true/false? When we move to practice, our goal is not only to get an answer. Our goal is to explain the thinking: what information we used, what step happened next, why the result is correct, and what we changed after testing.

Let's Practice

Let's practice with these activities:

Activity 1: Trace The Expression Type: individual Time: 10 minutes Instruction: Trace values before running Java. Student task: - Identify the data type of each variable. - Write one valid value for each data type. - Explain why average uses double. Code / model:

```
String name = "Ana";
int age = 18;
double average = 91.5;
boolean passed = true;
```

Output: Submit a trace table with expression and result.

Activity 2: Predict The Output Type: individual Time: 8 minutes Instruction: Predict what the program would display if we printed each result. Student task: - Write the expected output for each variable. - Check if the output is a number, text, or true/false. - Correct one prediction after tracing. Output: Submit predicted output lines.

Activity 3: Fix The Operator Error Type: pair Time: 10 minutes Instruction: Find and correct an operator mistake. Student task: - Check if = or == is needed. - Check if parentheses are missing. - Check if the data type fits the result. - Rewrite the corrected expression. Code / model:

```
int grade = 90;
boolean passed = grade = 75;
double average = 90 + 85 + 88 / 3.0;
```

Output: Submit corrected expressions and rule explanations.

Activity 4: Complete The Java Expression Type: hands-on coding Time: 12 minutes Instruction: Fill in missing operators to make the expression work. Student task: - Complete `sum = a ____ b;`. - Complete `passed = grade ____ 75;`. - Complete `valid = passed ____ present;`. - Run or trace the completed expressions. Output: Submit completed expressions and results.

Activity 5: Exit Ticket Type: exit ticket Time: 5 minutes Instruction: Review operator use. Student task: - What does = do? - What does == do? - When should we use parentheses? Output: Submit three answers.

Now We Apply

Now we apply it through these tasks: 1. Create variables for a student profile. 2. Write expressions for area, total price, and average. 3. Explain one expression step by step.

Challenge Task

Build a Grade Status Checker expression set: compute average, determine passing status, and display the result.

How Our Work Will Be Checked

Criteria - 20 points total - Correctness (5): Output or answer follows the required concept and instructions. - Completeness (4): All required parts are present. - Logic and sequence (4): Steps, code, diagram, or pseudocode follow a clear order. - Explanation (3): Student can explain what the work does and why it is correct. - Neatness/readability (2): Work is easy to read, label, and check. - Effort and revision (2): Student improves the work after feedback.

Let's Reflect

Let's reflect: - What concept from this lesson is clearest to you now? - What mistake did you notice or correct during practice? - Which activity helped you understand the lesson best? - How can this skill help you design or write a Java program? - What do you still need to practice before the next lesson?

How We Show Learning

Evidence we will use to check learning: - Correct data types and operators. - Expression results are accurate. - Student can explain calculations or conditions. - Completed guided-practice work with visible corrections or annotations. - A short explanation connecting the output to the lesson target.

Student Activities

Identify the Data Type. Predict the Output. Fix the Operator Error. Complete the Java Expression. Simple Calculator Practice. Trace Boolean Results. Create variables for a student profile. Write expressions for area, total price, and average. Explain one expression step by step.

Resources / References

JDK, IDE, projector or screenshots, printed worksheets, notebook, sample code snippets, and teacher-created answer guides.

LESSON 18

Lesson 18: Arithmetic And Assignment Operators In Java

What We'll Learn

By the end of this lesson, we can: - Choose suitable data types. - Use arithmetic, assignment, comparison, or logical operators. - Predict and explain expression results.

Words We'll Use

- Variable - named storage.
- Data type - kind of value.
- Operator - symbol that performs an action.
- Expression - code that produces a value.
- Boolean expression - true/false result.

Before We Start

What information about a student can a program store, and what operation might the program perform on it?

Let's Understand

Java operator tracing guide Let's work through Lesson 18: Arithmetic And Assignment Operators In Java together. Our focus is using variables, data types, and operators to build Java expressions. We will move from meaning, to examples, to practice, then to a task we can explain. What this lesson means: - Variables store values so a program can use them later. - Data types tell Java what kind of value we are storing, such as int, double, char, boolean, or String. - Operators are symbols that perform actions on values. - Arithmetic operators compute numbers. Comparison operators produce true or false. - Logical operators combine conditions so programs can make better decisions. - We trace expressions carefully because order and parentheses can change the result. How we study it step by step: - First, we connect the lesson to a familiar situation: What information about a student can a program store, and what operation might the program perform on it? - Next, we name the important parts so everyone uses the same vocabulary. - Then, we study Variables and ask: what does it do, where do we see it, and how can we check it? - Then, we study Data types and ask: what does it do, where do we see it, and how can we check it? - Then, we study Operators and ask: what does it do, where do we see it, and how can we check it? - Then, we study Expressions and ask: what does it do, where do we see it, and how can we check it? - After that, we compare a correct example with a common wrong version. - We trace the example slowly before running, drawing, or submitting anything. - Finally, we explain the result in our own words so the activity is not just copying. Rules and patterns we should remember: - Rule: Use = to assign a value and == to compare equality. Example: `score = 90;` stores 90, while `score == 90` asks if score is equal to 90. Check: Are we storing a value or asking a true/false question? - Rule: Use +, -, *, /, and % for arithmetic. Example: `int remainder = 10 % 3;` stores 1 because 10 divided by 3 leaves 1. Check: What does % give us? - Rule: Use parentheses when we want a calculation to happen first. Example: `average = (g1 + g2 + g3) / 3.0;` adds first, then divides. Check: What happens first inside the parentheses? - Rule: Use && when both conditions must be true. Example: `grade >= 75 && attendance >= 80` is true only when both checks pass. Check: What if attendance is only 70? - Rule: Use || when at least one condition may be true. Example: `hasPermit || isTeacherApproved` is true if either side is true. Check: How many sides need to be true? - Rule: Use ! to reverse a boolean condition. Example: `!isAbsent` means the student is not absent. Check: If `isAbsent` is false, what is `!isAbsent`? Common mistakes we will watch for: - Using String for values that should be numeric. - Confusing assignment = with comparison ==. - Forgetting parentheses in average formulas. - Expecting integer division to keep decimals. - Reading && and || without checking both sides of the condition. Mini-check before practice: - What data type fits this value? - Are we assigning a value or comparing two values? - Which operation happens first? - Does the final expression produce a number or true/false? When we move to practice, our goal is not only to get an answer. Our goal is to explain the thinking: what information we used, what step happened next, why the result is correct, and what we changed after testing.

Let's Practice

Let's practice with these activities:

Activity 1: Trace The Expression Type: individual Time: 10 minutes Instruction: Trace values before running Java. Student task: - Compute sum. - Compute remainder. - Compute average. - Explain why parentheses help. Code / model:

```
int a = 10;
int b = 3;
int sum = a + b;
int remainder = a % b;
double average = (90 + 85 + 88) / 3.0;
```

Output: Submit a trace table with expression and result.

Activity 2: Predict The Output Type: individual Time: 8 minutes Instruction: Predict what the program would display if we printed each result. Student task: - Write the expected output for each variable. - Check if the output is a number, text, or true/false. - Correct one prediction after tracing. Output: Submit predicted output lines.

Activity 3: Fix The Operator Error Type: pair Time: 10 minutes Instruction: Find and correct an operator mistake. Student task: - Check if = or == is needed. - Check if parentheses are missing. - Check if the data type fits the result. - Rewrite the corrected expression. Code / model:

```
int grade = 90;
boolean passed = grade = 75;
double average = 90 + 85 + 88 / 3.0;
```

Output: Submit corrected expressions and rule explanations.

Activity 4: Complete The Java Expression Type: hands-on coding Time: 12 minutes Instruction: Fill in missing operators to make the expression work. Student task: - Complete `sum = a ____ b;`. - Complete `passed = grade ____ 75;`. - Complete `valid = passed ____ present;`. - Run or trace the completed expressions. Output: Submit completed expressions and results.

Activity 5: Exit Ticket Type: exit ticket Time: 5 minutes Instruction: Review operator use. Student task: - What does = do? - What does == do? - When should we use parentheses? Output: Submit three answers.

Now We Apply

Now we apply it through these tasks: 1. Create variables for a student profile. 2. Write expressions for area, total price, and average. 3. Explain one expression step by step.

Challenge Task

Build a Grade Status Checker expression set: compute average, determine passing status, and display the result.

How Our Work Will Be Checked

Criteria - 20 points total - Correctness (5): Output or answer follows the required concept and instructions. - Completeness (4): All required parts are present. - Logic and sequence (4): Steps, code, diagram, or pseudocode follow a clear order. - Explanation (3): Student can explain what the work does and why it is correct. - Neatness/readability (2): Work is easy to read, label, and check. - Effort and revision (2): Student improves the work after feedback.

Let's Reflect

Let's reflect: - What concept from this lesson is clearest to you now? - What mistake did you notice or correct during practice? - Which activity helped you understand the lesson best? - How can this skill help you design or write a Java program? - What do you still need to practice before the next lesson?

How We Show Learning

Evidence we will use to check learning: - Correct data types and operators. - Expression results are accurate. - Student can explain calculations or conditions. - Completed guided-practice work with visible corrections or annotations. - A short explanation connecting the output to the lesson target.

Student Activities

Identify the Data Type. Predict the Output. Fix the Operator Error. Complete the Java Expression. Simple Calculator Practice. Trace Boolean Results. Create variables for a student profile. Write expressions for area, total price, and average. Explain one expression step by step.

Resources / References

JDK, IDE, projector or screenshots, printed worksheets, notebook, sample code snippets, and teacher-created answer guides.

LESSON 19

Lesson 19: Comparison And Logical Operators In Java

What We'll Learn

By the end of this lesson, we can: - Choose suitable data types. - Use arithmetic, assignment, comparison, or logical operators. - Predict and explain expression results.

Words We'll Use

- Variable - named storage.
- Data type - kind of value.
- Operator - symbol that performs an action.
- Expression - code that produces a value.
- Boolean expression - true/false result.

Before We Start

What information about a student can a program store, and what operation might the program perform on it?

Let's Understand

Java operator tracing guide Let's work through Lesson 19: Comparison And Logical Operators In Java together. Our focus is using variables, data types, and operators to build Java expressions. We will move from meaning, to examples, to practice, then to a task we can explain. What this lesson means: - Variables store values so a program can use them later. - Data types tell Java what kind of value we are storing, such as int, double, char, boolean, or String. - Operators are symbols that perform actions on values. - Arithmetic operators compute numbers. Comparison operators produce true or false. - Logical operators combine conditions so programs can make better decisions. - We trace expressions carefully because order and parentheses can change the result. How we study it step by step: - First, we connect the lesson to a familiar situation: What information about a student can a program store, and what operation might the program perform on it? - Next, we name the important parts so everyone uses the same vocabulary. - Then, we study Variables and ask: what does it do, where do we see it, and how can we check it? - Then, we study Data types and ask: what does it do, where do we see it, and how can we check it? - Then, we study Operators and ask: what does it do, where do we see it, and how can we check it? - Then, we study Expressions and ask: what does it do, where do we see it, and how can we check it? - After that, we compare a correct example with a common wrong version. - We trace the example slowly before running, drawing, or submitting anything. - Finally, we explain the result in our own words so the activity is not just copying. Rules and patterns we should remember: - Rule: Use = to assign a value and == to compare equality. Example: `score = 90;` stores 90, while `score == 90` asks if score is equal to 90. Check: Are we storing a value or asking a true/false question? - Rule: Use +, -, *, /, and % for arithmetic. Example: `int remainder = 10 % 3;` stores 1 because 10 divided by 3 leaves 1. Check: What does % give us? - Rule: Use parentheses when we want a calculation to happen first. Example: `average = (g1 + g2 + g3) / 3.0;` adds first, then divides. Check: What happens first inside the parentheses? - Rule: Use && when both conditions must be true. Example: `grade >= 75 && attendance >= 80` is true only when both checks pass. Check: What if attendance is only 70? - Rule: Use || when at least one condition may be true. Example: `hasPermit || isTeacherApproved` is true if either side is true. Check: How many sides need to be true? - Rule: Use ! to reverse a boolean condition. Example: `!isAbsent` means the student is not absent. Check: If `isAbsent` is false, what is `!isAbsent`? Common mistakes we will watch for: - Using String for values that should be numeric. - Confusing assignment = with comparison ==. - Forgetting parentheses in average formulas. - Expecting integer division to keep decimals. - Reading && and || without checking both sides of the condition. Mini-check before practice: - What data type fits this value? - Are we assigning a value or comparing two values? - Which operation happens first? - Does the final expression produce a number or true/false? When we move to practice, our goal is not only to get an answer. Our goal is to explain the thinking: what information we used, what step happened next, why the result is correct, and what we changed after testing.

Let's Practice

Let's practice with these activities:

Activity 1: Trace The Expression Type: individual Time: 10 minutes Instruction: Trace values before running Java. Student task: - Evaluate passed. - Evaluate canReceiveCertificate. - Change present to false and trace again. Code / model:

```
int grade = 82;
boolean present = true;
boolean passed = grade >= 75;
boolean canReceiveCertificate = passed && present;
```

Output: Submit a trace table with expression and result.

Activity 2: Predict The Output Type: individual Time: 8 minutes Instruction: Predict what the program would display if we printed each result. Student task: - Write the expected output for each variable. - Check if the output is a number, text, or true/false. - Correct one prediction after tracing. Output: Submit predicted output lines.

Activity 3: Fix The Operator Error Type: pair Time: 10 minutes Instruction: Find and correct an operator mistake. Student task: - Check if = or == is needed. - Check if parentheses are missing. - Check if the data type fits the result. - Rewrite the corrected expression. Code / model:

```
int grade = 90;
boolean passed = grade = 75;
double average = 90 + 85 + 88 / 3.0;
```

Output: Submit corrected expressions and rule explanations.

Activity 4: Complete The Java Expression Type: hands-on coding Time: 12 minutes Instruction: Fill in missing operators to make the expression work. Student task: - Complete `sum = a ____ b;`. - Complete `passed = grade ____ 75;`. - Complete `valid = passed ____ present;`. - Run or trace the completed expressions. Output: Submit completed expressions and results.

Activity 5: Exit Ticket Type: exit ticket Time: 5 minutes Instruction: Review operator use. Student task: - What does = do? - What does == do? - When should we use parentheses? Output: Submit three answers.

Now We Apply

Now we apply it through these tasks: 1. Create variables for a student profile. 2. Write expressions for area, total price, and average. 3. Explain one expression step by step.

Challenge Task

Build a Grade Status Checker expression set: compute average, determine passing status, and display the result.

How Our Work Will Be Checked

Criteria - 20 points total - Correctness (5): Output or answer follows the required concept and instructions. - Completeness (4): All required parts are present. - Logic and sequence (4): Steps, code, diagram, or pseudocode follow a clear order. - Explanation (3): Student can explain what the work does and why it is correct. - Neatness/readability (2): Work is easy to read, label, and check. - Effort and revision (2): Student improves the work after feedback.

Let's Reflect

Let's reflect: - What concept from this lesson is clearest to you now? - What mistake did you notice or correct during practice? - Which activity helped you understand the lesson best? - How can this skill help you design or write a Java program? - What do you still need to practice before the next lesson?

How We Show Learning

Evidence we will use to check learning: - Correct data types and operators. - Expression results are accurate. - Student can explain calculations or conditions. - Completed guided-practice work with visible corrections or annotations. - A short explanation connecting the output to the lesson target.

Student Activities

Identify the Data Type. Predict the Output. Fix the Operator Error. Complete the Java Expression. Simple Calculator Practice. Trace Boolean Results. Create variables for a student profile. Write expressions for area, total price, and average. Explain one expression step by step.

Resources / References

JDK, IDE, projector or screenshots, printed worksheets, notebook, sample code snippets, and teacher-created answer guides.

LESSON 20

Lesson 20: Building And Tracing Java Expressions

What We'll Learn

By the end of this lesson, we can: - Choose suitable data types. - Use arithmetic, assignment, comparison, or logical operators. - Predict and explain expression results.

Words We'll Use

- Variable - named storage.
- Data type - kind of value.
- Operator - symbol that performs an action.
- Expression - code that produces a value.
- Boolean expression - true/false result.

Before We Start

What information about a student can a program store, and what operation might the program perform on it?

Let's Understand

Java operator tracing guide Let's work through Lesson 20: Building And Tracing Java Expressions together. Our focus is using variables, data types, and operators to build Java expressions. We will move from meaning, to examples, to practice, then to a task we can explain. What this lesson means: - Variables store values so a program can use them later. - Data types tell Java what kind of value we are storing, such as int, double, char, boolean, or String. - Operators are symbols that perform actions on values. - Arithmetic operators compute numbers. Comparison operators produce true or false. - Logical operators combine conditions so programs can make better decisions. - We trace expressions carefully because order and parentheses can change the result. How we study it step by step: - First, we connect the lesson to a familiar situation: What information about a student can a program store, and what operation might the program perform on it? - Next, we name the important parts so everyone uses the same vocabulary. - Then, we study Variables and ask: what does it do, where do we see it, and how can we check it? - Then, we study Data types and ask: what does it do, where do we see it, and how can we check it? - Then, we study Operators and ask: what does it do, where do we see it, and how can we check it? - Then, we study Expressions and ask: what does it do, where do we see it, and how can we check it? - After that, we compare a correct example with a common wrong version. - We trace the example slowly before running, drawing, or submitting anything. - Finally, we explain the result in our own words so the activity is not just copying. Rules and patterns we should remember: - Rule: Use = to assign a value and == to compare equality. Example: `score = 90;` stores 90, while `score == 90` asks if score is equal to 90. Check: Are we storing a value or asking a true/false question? - Rule: Use +, -, *, /, and % for arithmetic. Example: `int remainder = 10 % 3;` stores 1 because 10 divided by 3 leaves 1. Check: What does % give us? - Rule: Use parentheses when we want a calculation to happen first. Example: `average = (g1 + g2 + g3) / 3.0;` adds first, then divides. Check: What happens first inside the parentheses? - Rule: Use && when both conditions must be true. Example: `grade >= 75 && attendance >= 80` is true only when both checks pass. Check: What if attendance is only 70? - Rule: Use || when at least one condition may be true. Example: `hasPermit || isTeacherApproved` is true if either side is true. Check: How many sides need to be true? - Rule: Use ! to reverse a boolean condition. Example: `!isAbsent` means the student is not absent. Check: If `isAbsent` is false, what is `!isAbsent`? Common mistakes we will watch for: - Using String for values that should be numeric. - Confusing assignment = with comparison ==. - Forgetting parentheses in average formulas. - Expecting integer division to keep decimals. - Reading && and || without checking both sides of the condition. Mini-check before practice: - What data type fits this value? - Are we assigning a value or comparing two values? - Which operation happens first? - Does the final expression produce a number or true/false? When we move to practice, our goal is not only to get an answer. Our goal is to explain the thinking: what information we used, what step happened next, why the result is correct, and what we changed after testing.

Let's Practice

Let's practice with these activities:

Activity 1: Trace The Expression Type: individual Time: 10 minutes Instruction: Trace values before running Java. Student task: - Trace total. - Trace hasDiscount. - Trace finalTotal. - Explain the expression order. Code / model:

```
double price = 25.50;
int quantity = 4;
double total = price * quantity;
boolean hasDiscount = total >= 100;
double finalTotal = hasDiscount ? total - 10 : total;
```

Output: Submit a trace table with expression and result.

Activity 2: Predict The Output Type: individual Time: 8 minutes Instruction: Predict what the program would display if we printed each result. Student task: - Write the expected output for each variable. - Check if the output is a number, text, or true/false. - Correct one prediction after tracing. Output: Submit predicted output lines.

Activity 3: Fix The Operator Error Type: pair Time: 10 minutes Instruction: Find and correct an operator mistake. Student task: - Check if = or == is needed. - Check if parentheses are missing. - Check if the data type fits the result. - Rewrite the corrected expression. Code / model:

```
int grade = 90;
boolean passed = grade = 75;
double average = 90 + 85 + 88 / 3.0;
```

Output: Submit corrected expressions and rule explanations.

Activity 4: Complete The Java Expression Type: hands-on coding Time: 12 minutes Instruction: Fill in missing operators to make the expression work. Student task: - Complete `sum = a ____ b;`. - Complete `passed = grade ____ 75;`. - Complete `valid = passed ____ present;`. - Run or trace the completed expressions. Output: Submit completed expressions and results.

Activity 5: Exit Ticket Type: exit ticket Time: 5 minutes Instruction: Review operator use. Student task: - What does = do? - What does == do? - When should we use parentheses? Output: Submit three answers.

Now We Apply

Now we apply it through these tasks: 1. Create variables for a student profile. 2. Write expressions for area, total price, and average. 3. Explain one expression step by step.

Challenge Task

Build a Grade Status Checker expression set: compute average, determine passing status, and display the result.

How Our Work Will Be Checked

Criteria - 20 points total - Correctness (5): Output or answer follows the required concept and instructions. - Completeness (4): All required parts are present. - Logic and sequence (4): Steps, code, diagram, or pseudocode follow a clear order. - Explanation (3): Student can explain what the work does and why it is correct. - Neatness/readability (2): Work is easy to read, label, and check. - Effort and revision (2): Student improves the work after feedback.

Let's Reflect

Let's reflect: - What concept from this lesson is clearest to you now? - What mistake did you notice or correct during practice? - Which activity helped you understand the lesson best? - How can this skill help you design or write a Java program? - What do you still need to practice before the next lesson?

How We Show Learning

Evidence we will use to check learning: - Correct data types and operators. - Expression results are accurate. - Student can explain calculations or conditions. - Completed guided-practice work with visible corrections or annotations. - A short explanation connecting the output to the lesson target.

Student Activities

Identify the Data Type. Predict the Output. Fix the Operator Error. Complete the Java Expression. Simple Calculator Practice. Trace Boolean Results. Create variables for a student profile. Write expressions for area, total price, and average. Explain one expression step by step.

Resources / References

JDK, IDE, projector or screenshots, printed worksheets, notebook, sample code snippets, and teacher-created answer guides.

LESSON 21

Lesson 21: Reading User Input With Scanner

What We'll Learn

By the end of this lesson, we can: - Use Java I/O concepts appropriately. - Create readable program output. - Test and explain input/output or file behavior.

Words We'll Use

- Scanner - Java class for input.
- Prompt - message asking for input.
- Output formatting - arranging results clearly.
- FileWriter - class for writing text.
- File - stored data on a computer.

Before We Start

Why is a program more useful if it can ask, display, save, or read information?

Let's Understand

Scanner input flow guide Let's work through Lesson 21: Reading User Input With Scanner together. Our focus is reading input, displaying output, and saving or reading simple text files. We will move from meaning, to examples, to practice, then to a task we can explain. What this lesson means: - Input lets a Java program receive information instead of using fixed values only. - Output lets a Java program communicate clear results to the user. - Scanner can read values typed by the user or lines from a file. - FileWriter can save text into a file so the data remains after the program runs. - File I/O needs careful testing because file names, file locations, and errors matter. - Clean prompts and labeled output make our programs easier to use and check. How we study it step by step: - First, we connect the lesson to a familiar situation: Why is a program more useful if it can ask, display, save, or read information? - Next, we name the important parts so everyone uses the same vocabulary. - Then, we study Scanner and ask: what does it do, where do we see it, and how can we check it? - Then, we study Output formatting and ask: what does it do, where do we see it, and how can we check it? - Then, we study File writing and ask: what does it do, where do we see it, and how can we check it? - Then, we study File reading and ask: what does it do, where do we see it, and how can we check it? - After that, we compare a correct example with a common wrong version. - We trace the example slowly before running, drawing, or submitting anything. - Finally, we explain the result in our own words so the activity is not just copying. Rules and patterns we should remember: - Rule: Import the class before using Scanner, FileWriter, or File. Example: `import java.util.Scanner;` is needed before creating a Scanner for keyboard input. Check: Which import do we need for Scanner? - Rule: Show a clear prompt before reading user input. Example: `System.out.print("Enter your name: ");` tells the user what to type. Check: What should the user enter? - Rule: Store input in a variable with a suitable data type. Example: `String name = input.nextLine();` stores text, while `int age = input.nextInt();` stores a whole number. Check: Which method fits a name? - Rule: Label output so the user understands what the value means. Example: `System.out.println("Average: " + average);` is clearer than printing only `average`. Check: What label helps us read the result? - Rule: Close file writers or readers when we are done. Example: `writer.close();` finishes writing and releases the file. Check: What line tells Java we are done with the file? - Rule: Use simple error handling when a file might not exist or writing might fail. Example: `catch (IOException error)` lets us display a helpful message instead of crashing silently. Check: What message should the user see if saving fails? Common mistakes we will watch for: - Forgetting the import statement. - Asking for input without a clear prompt. - Reading the wrong data type. - Saving a file but not knowing where it was created. - Displaying unlabeled output that is hard to check. Mini-check before practice: - What prompt will the user see? - What variable stores the input? - What output should appear? - If a file is used, where is it saved or read from? When we move to practice, our goal is not only to get an answer. Our goal is to explain the thinking: what information we used, what step happened next, why the result is correct, and what we changed after testing.

Let's Practice

Let's practice with these activities:

Activity 1: I/O Code Walkthrough Type: individual Time: 10 minutes Instruction: Read the model and label the important input/output or file parts. Student task: - Find the import statement. - Find the prompt. - Find where input is stored. - Change the variable name safely. Code / model:

```
import java.util.Scanner;

public class InputExample {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.print("Enter your name: ");
        String name = input.nextLine();
        System.out.println("Hello, " + name + "!");
    }
}
```

Output: Submit labeled code parts and one explanation.

Activity 2: Predict The Program Behavior Type: pair Time: 8 minutes Instruction: Predict what the user sees or what file action happens. Student task: - Write what appears before input. - Write what value is stored or saved. - Write what output or file content should appear. - Compare with the code model. Output: Submit predicted behavior in three steps.

Activity 3: Complete The Missing Line Type: hands-on coding Time: 12 minutes Instruction: Complete one missing Java I/O line. Student task: - Add the needed import. - Add a prompt. - Add an input/read/write line. - Add a clear output line. Code / model:

```
// Complete the missing lines for this lesson focus.
// import goes here
public class PracticeIO {
    public static void main(String[] args) {
        // prompt, read/write, and output go here
    }
}
```

Output: Submit completed code or completed missing lines.

Activity 4: Test Evidence Type: individual Time: 10 minutes Instruction: Gather evidence that the program behavior is correct. Student task: - Run or trace with sample data. - Record input used. - Record console output. - If a file is involved, record file name and expected contents. Output: Submit input, output, and file evidence notes.

Activity 5: Exit Ticket Type: exit ticket Time: 5 minutes Instruction: Review Java I/O habits. Student task: - Why do prompts matter? - Why should output be labeled? - Why do file programs need testing? Output: Submit three answers.

Now We Apply

Now we apply it through these tasks: 1. Create a student introduction program. 2. Create a receipt or grade report output. 3. Save and/or read student information from a text file.

Challenge Task

Create a Student Information File Java program that reads user input, displays a clean summary, saves the information to a text file, and includes a written explanation.

How Our Work Will Be Checked

Criteria - 20 points total - Correctness (5): Output or answer follows the required concept and instructions. - Completeness (4): All required parts are present. - Logic and sequence (4): Steps, code, diagram, or pseudocode follow a clear order. - Explanation (3): Student can explain what the work does and why it is correct. - Neatness/readability (2): Work is easy to read, label, and check. - Effort and revision (2): Student improves the work after feedback.

Let's Reflect

Let's reflect: - What concept from this lesson is clearest to you now? - What mistake did you notice or correct during practice? - Which activity helped you understand the lesson best? - How can this skill help you design or write a Java program? - What do you still need to practice before the next lesson?

How We Show Learning

Evidence we will use to check learning: - Program reads required input. - Program displays clean output. - Program saves or reads a text file. - Student provides evidence and explanation. - Completed guided-practice work with visible corrections or annotations. - A short explanation connecting the output to the lesson target.

Student Activities

Input Practice. Output Formatting Practice. File I/O Concept Check. Trace File Writing Steps. Read Student Info from a File. Peer Test and Feedback. Create a student introduction program. Create a receipt or grade report output. Save and/or read student information from a text file.

Resources / References

JDK, IDE, projector or screenshots, printed worksheets, notebook, sample code snippets, and teacher-created answer guides.

LESSON 22

Lesson 22: Formatting Clear Java Program Output

What We'll Learn

By the end of this lesson, we can: - Use Java I/O concepts appropriately. - Create readable program output. - Test and explain input/output or file behavior.

Words We'll Use

- Scanner - Java class for input.
- Prompt - message asking for input.
- Output formatting - arranging results clearly.
- FileWriter - class for writing text.
- File - stored data on a computer.

Before We Start

Why is a program more useful if it can ask, display, save, or read information?

Let's Understand

Scanner input flow guide Let's work through Lesson 22: Formatting Clear Java Program Output together. Our focus is reading input, displaying output, and saving or reading simple text files. We will move from meaning, to examples, to practice, then to a task we can explain. What this lesson means: - Input lets a Java program receive information instead of using fixed values only. - Output lets a Java program communicate clear results to the user. - Scanner can read values typed by the user or lines from a file. - FileWriter can save text into a file so the data remains after the program runs. - File I/O needs careful testing because file names, file locations, and errors matter. - Clean prompts and labeled output make our programs easier to use and check. How we study it step by step: - First, we connect the lesson to a familiar situation: Why is a program more useful if it can ask, display, save, or read information? - Next, we name the important parts so everyone uses the same vocabulary. - Then, we study Scanner and ask: what does it do, where do we see it, and how can we check it? - Then, we study Output formatting and ask: what does it do, where do we see it, and how can we check it? - Then, we study File writing and ask: what does it do, where do we see it, and how can we check it? - Then, we study File reading and ask: what does it do, where do we see it, and how can we check it? - After that, we compare a correct example with a common wrong version. - We trace the example slowly before running, drawing, or submitting anything. - Finally, we explain the result in our own words so the activity is not just copying. Rules and patterns we should remember: - Rule: Import the class before using Scanner, FileWriter, or File. Example: `import java.util.Scanner;` is needed before creating a Scanner for keyboard input. Check: Which import do we need for Scanner? - Rule: Show a clear prompt before reading user input. Example: `System.out.print("Enter your name: ");` tells the user what to type. Check: What should the user enter? - Rule: Store input in a variable with a suitable data type. Example: `String name = input.nextLine();` stores text, while `int age = input.nextInt();` stores a whole number. Check: Which method fits a name? - Rule: Label output so the user understands what the value means. Example: `System.out.println("Average: " + average);` is clearer than printing only `average`. Check: What label helps us read the result? - Rule: Close file writers or readers when we are done. Example: `writer.close();` finishes writing and releases the file. Check: What line tells Java we are done with the file? - Rule: Use simple error handling when a file might not exist or writing might fail. Example: `catch (IOException error)` lets us display a helpful message instead of crashing silently. Check: What message should the user see if saving fails? Common mistakes we will watch for: - Forgetting the import statement. - Asking for input without a clear prompt. - Reading the wrong data type. - Saving a file but not knowing where it was created. - Displaying unlabeled output that is hard to check. Mini-check before practice: - What prompt will the user see? - What variable stores the input? - What output should appear? - If a file is used, where is it saved or read from? When we move to practice, our goal is not only to get an answer. Our goal is to explain the thinking: what information we used, what step happened next, why the result is correct, and what we changed after testing.

Let's Practice

Let's practice with these activities:

Activity 1: I/O Code Walkthrough Type: individual Time: 10 minutes Instruction: Read the model and label the important input/output or file parts. Student task: - Label each output. - Improve one output message. - Predict the exact console display. Code / model:

```
String name = "Ana";
int grade = 95;
System.out.println("Student: " + name);
System.out.println("Grade: " + grade);
```

Output: Submit labeled code parts and one explanation.

Activity 2: Predict The Program Behavior Type: pair Time: 8 minutes Instruction: Predict what the user sees or what file action happens. Student task: - Write what appears before input. - Write what value is stored or saved. - Write what output or file content should appear. - Compare with the code model. Output: Submit predicted behavior in three steps.

Activity 3: Complete The Missing Line Type: hands-on coding Time: 12 minutes Instruction: Complete one missing Java I/O line. Student task: - Add the needed import. - Add a prompt. - Add an input/read/write line. - Add a clear output line. Code / model:

```
// Complete the missing lines for this lesson focus.
// import goes here
public class PracticeIO {
    public static void main(String[] args) {
        // prompt, read/write, and output go here
    }
}
```

Output: Submit completed code or completed missing lines.

Activity 4: Test Evidence Type: individual Time: 10 minutes Instruction: Gather evidence that the program behavior is correct. Student task: - Run or trace with sample data. - Record input used. - Record console output. - If a file is involved, record file name and expected contents. Output: Submit input, output, and file evidence notes.

Activity 5: Exit Ticket Type: exit ticket Time: 5 minutes Instruction: Review Java I/O habits. Student task: - Why do prompts matter? - Why should output be labeled? - Why do file programs need testing? Output: Submit three answers.

Now We Apply

Now we apply it through these tasks: 1. Create a student introduction program. 2. Create a receipt or grade report output. 3. Save and/or read student information from a text file.

Challenge Task

Create a Student Information File Java program that reads user input, displays a clean summary, saves the information to a text file, and includes a written explanation.

How Our Work Will Be Checked

Criteria - 20 points total - Correctness (5): Output or answer follows the required concept and instructions. - Completeness (4): All required parts are present. - Logic and sequence (4): Steps, code, diagram, or pseudocode follow a clear order. - Explanation (3): Student can explain what the work does and why it is correct. - Neatness/readability (2): Work is easy to read, label, and check. - Effort and revision (2): Student improves the work after feedback.

Let's Reflect

Let's reflect: - What concept from this lesson is clearest to you now? - What mistake did you notice or correct during practice? - Which activity helped you understand the lesson best? - How can this skill help you design or write a Java program? - What do you still need to practice before the next lesson?

How We Show Learning

Evidence we will use to check learning: - Program reads required input. - Program displays clean output. - Program saves or reads a text file. - Student provides evidence and explanation. - Completed guided-practice work with visible corrections or annotations. - A short explanation connecting the output to the lesson target.

Student Activities

Input Practice. Output Formatting Practice. File I/O Concept Check. Trace File Writing Steps. Read Student Info from a File. Peer Test and Feedback. Create a student introduction program. Create a receipt or grade report output. Save and/or read student information from a text file.

Resources / References

JDK, IDE, projector or screenshots, printed worksheets, notebook, sample code snippets, and teacher-created answer guides.

LESSON 23

Lesson 23: Writing Text Data To Files In Java

What We'll Learn

By the end of this lesson, we can: - Use Java I/O concepts appropriately. - Create readable program output. - Test and explain input/output or file behavior.

Words We'll Use

- Scanner - Java class for input.
- Prompt - message asking for input.
- Output formatting - arranging results clearly.
- FileWriter - class for writing text.
- File - stored data on a computer.

Before We Start

Why is a program more useful if it can ask, display, save, or read information?

Let's Understand

Java file input and output flow guide Let's work through Lesson 23: Writing Text Data To Files In Java together. Our focus is reading input, displaying output, and saving or reading simple text files. We will move from meaning, to examples, to practice, then to a task we can explain. What this lesson means: - Input lets a Java program receive information instead of using fixed values only. - Output lets a Java program communicate clear results to the user. - Scanner can read values typed by the user or lines from a file. - FileWriter can save text into a file so the data remains after the program runs. - File I/O needs careful testing because file names, file locations, and errors matter. - Clean prompts and labeled output make our programs easier to use and check. How we study it step by step: - First, we connect the lesson to a familiar situation: Why is a program more useful if it can ask, display, save, or read information? - Next, we name the important parts so everyone uses the same vocabulary. - Then, we study Scanner and ask: what does it do, where do we see it, and how can we check it? - Then, we study Output formatting and ask: what does it do, where do we see it, and how can we check it? - Then, we study File writing and ask: what does it do, where do we see it, and how can we check it? - Then, we study File reading and ask: what does it do, where do we see it, and how can we check it? - After that, we compare a correct example with a common wrong version. - We trace the example slowly before running, drawing, or submitting anything. - Finally, we explain the result in our own words so the activity is not just copying. Rules and patterns we should remember: - Rule: Import the class before using Scanner, FileWriter, or File. Example: `import java.util.Scanner;` is needed before creating a Scanner for keyboard input. Check: Which import do we need for Scanner? - Rule: Show a clear prompt before reading user input. Example: `System.out.print("Enter your name: ");` tells the user what to type. Check: What should the user enter? - Rule: Store input in a variable with a suitable data type. Example: `String name = input.nextLine();` stores text, while `int age = input.nextInt();` stores a whole number. Check: Which method fits a name? - Rule: Label output so the user understands what the value means. Example: `System.out.println("Average: " + average);` is clearer than printing only `average`. Check: What label helps us read the result? - Rule: Close file writers or readers when we are done. Example: `writer.close();` finishes writing and releases the file. Check: What line tells Java we are done with the file? - Rule: Use simple error handling when a file might not exist or writing might fail. Example: `catch (IOException error)` lets us display a helpful message instead of crashing silently. Check: What message should the user see if saving fails? Common mistakes we will watch for: - Forgetting the import statement. - Asking for input without a clear prompt. - Reading the wrong data type. - Saving a file but not knowing where it was created. - Displaying unlabeled output that is hard to check. Mini-check before practice: - What prompt will the user see? - What variable stores the input? - What output should appear? - If a file is used, where is it saved or read from? When we move to practice, our goal is not only to get an answer. Our goal is to explain the thinking: what information we used, what step happened next, why the result is correct, and what we changed after testing.

Let's Practice

Let's practice with these activities:

Activity 1: I/O Code Walkthrough Type: individual Time: 10 minutes Instruction: Read the model and label the important input/output or file parts. Student task: - Find the FileWriter line. - Find the file name. - Find the write line. - Find the close line. Code / model:

```
import java.io.FileWriter;
import java.io.IOException;

public class WriteFileExample {
    public static void main(String[] args) {
        try {
            FileWriter writer = new FileWriter("student.txt");
            writer.write("Name: Ana\nCourse: CC 102");
            writer.close();
            System.out.println("File saved.");
        } catch (IOException error) {
            System.out.println("An error occurred.");
        }
    }
}
```

Output: Submit labeled code parts and one explanation.

Activity 2: Predict The Program Behavior Type: pair Time: 8 minutes Instruction: Predict what the user sees or what file action happens. Student task: - Write what appears before input. - Write what value is stored or saved. - Write what output or file content should appear. - Compare with the code model. Output: Submit predicted behavior in three steps.

Activity 3: Complete The Missing Line Type: hands-on coding Time: 12 minutes Instruction: Complete one missing Java I/O line. Student task: - Add the needed import. - Add a prompt. - Add an input/read/write line. - Add a clear output line. Code / model:

```
// Complete the missing lines for this lesson focus.
// import goes here
public class PracticeIO {
    public static void main(String[] args) {
        // prompt, read/write, and output go here
    }
}
```

Output: Submit completed code or completed missing lines.

Activity 4: Test Evidence Type: individual Time: 10 minutes Instruction: Gather evidence that the program behavior is correct. Student task: - Run or trace with sample data. - Record input used. - Record console output. - If a file is involved, record file name and expected contents. Output: Submit input, output, and file evidence notes.

Activity 5: Exit Ticket Type: exit ticket Time: 5 minutes Instruction: Review Java I/O habits. Student task: - Why do prompts matter? - Why should output be labeled? - Why do file programs need testing? Output: Submit three answers.

Now We Apply

Now we apply it through these tasks: 1. Create a student introduction program. 2. Create a receipt or grade report output. 3. Save and/or read student information from a text file.

Challenge Task

Create a Student Information File Java program that reads user input, displays a clean summary, saves the information to a text file, and includes a written explanation.

How Our Work Will Be Checked

Criteria - 20 points total - Correctness (5): Output or answer follows the required concept and instructions. - Completeness (4): All required parts are present. - Logic and sequence (4): Steps, code, diagram, or pseudocode follow a clear order. - Explanation (3): Student can explain what the work does and why it is correct. - Neatness/readability (2): Work is easy to read, label, and check. - Effort and revision (2): Student improves the work after feedback.

Let's Reflect

Let's reflect: - What concept from this lesson is clearest to you now? - What mistake did you notice or correct during practice? - Which activity helped you understand the lesson best? - How can this skill help you design or write a Java program? - What do you still need to practice before the next lesson?

How We Show Learning

Evidence we will use to check learning: - Program reads required input. - Program displays clean output. - Program saves or reads a text file. - Student provides evidence and explanation. - Completed guided-practice work with visible corrections or annotations. - A short explanation connecting the output to the lesson target.

Student Activities

Input Practice. Output Formatting Practice. File I/O Concept Check. Trace File Writing Steps. Read Student Info from a File. Peer Test and Feedback. Create a student introduction program. Create a receipt or grade report output. Save and/or read student information from a text file.

Resources / References

JDK, IDE, projector or screenshots, printed worksheets, notebook, sample code snippets, and teacher-created answer guides.

LESSON 24

Lesson 24: Reading Text Data From Files In Java

What We'll Learn

By the end of this lesson, we can: - Use Java I/O concepts appropriately. - Create readable program output. - Test and explain input/output or file behavior.

Words We'll Use

- Scanner - Java class for input.
- Prompt - message asking for input.
- Output formatting - arranging results clearly.
- FileWriter - class for writing text.
- File - stored data on a computer.

Before We Start

Why is a program more useful if it can ask, display, save, or read information?

Let's Understand

Java file input and output flow guide Let's work through Lesson 24: Reading Text Data From Files In Java together. Our focus is reading input, displaying output, and saving or reading simple text files. We will move from meaning, to examples, to practice, then to a task we can explain. What this lesson means: - Input lets a Java program receive information instead of using fixed values only. - Output lets a Java program communicate clear results to the user. - Scanner can read values typed by the user or lines from a file. - FileWriter can save text into a file so the data remains after the program runs. - File I/O needs careful testing because file names, file locations, and errors matter. - Clean prompts and labeled output make our programs easier to use and check. How we study it step by step: - First, we connect the lesson to a familiar situation: Why is a program more useful if it can ask, display, save, or read information? - Next, we name the important parts so everyone uses the same vocabulary. - Then, we study Scanner and ask: what does it do, where do we see it, and how can we check it? - Then, we study Output formatting and ask: what does it do, where do we see it, and how can we check it? - Then, we study File writing and ask: what does it do, where do we see it, and how can we check it? - Then, we study File reading and ask: what does it do, where do we see it, and how can we check it? - After that, we compare a correct example with a common wrong version. - We trace the example slowly before running, drawing, or submitting anything. - Finally, we explain the result in our own words so the activity is not just copying. Rules and patterns we should remember: - Rule: Import the class before using Scanner, FileWriter, or File. Example: `import java.util.Scanner;` is needed before creating a Scanner for keyboard input. Check: Which import do we need for Scanner? - Rule: Show a clear prompt before reading user input. Example: `System.out.print("Enter your name: ");` tells the user what to type. Check: What should the user enter? - Rule: Store input in a variable with a suitable data type. Example: `String name = input.nextLine();` stores text, while `int age = input.nextInt();` stores a whole number. Check: Which method fits a name? - Rule: Label output so the user understands what the value means. Example: `System.out.println("Average: " + average);` is clearer than printing only `average`. Check: What label helps us read the result? - Rule: Close file writers or readers when we are done. Example: `writer.close();` finishes writing and releases the file. Check: What line tells Java we are done with the file? - Rule: Use simple error handling when a file might not exist or writing might fail. Example: `catch (IOException error)` lets us display a helpful message instead of crashing silently. Check: What message should the user see if saving fails? Common mistakes we will watch for: - Forgetting the import statement. - Asking for input without a clear prompt. - Reading the wrong data type. - Saving a file but not knowing where it was created. - Displaying unlabeled output that is hard to check. Mini-check before practice: - What prompt will the user see? - What variable stores the input? - What output should appear? - If a file is used, where is it saved or read from? When we move to practice, our goal is not only to get an answer. Our goal is to explain the thinking: what information we used, what step happened next, why the result is correct, and what we changed after testing.

Let's Practice

Let's practice with these activities:

Activity 1: I/O Code Walkthrough Type: individual Time: 10 minutes Instruction: Read the model and label the important input/output or file parts. Student task: - Find the File object. - Find the Scanner reader. - Find the loop that reads lines. - Explain what happens if the file is missing. Code / model:

```
import java.io.File;
import java.io.FileNotFoundException;
import java.util.Scanner;

public class ReadFileExample {
    public static void main(String[] args) {
        try {
            File file = new File("student.txt");
            Scanner reader = new Scanner(file);
            while (reader.hasNextLine()) {
                System.out.println(reader.nextLine());
            }
            reader.close();
        } catch (FileNotFoundException error) {
            System.out.println("File not found.");
        }
    }
}
```

Output: Submit labeled code parts and one explanation.

Activity 2: Predict The Program Behavior Type: pair Time: 8 minutes Instruction: Predict what the user sees or what file action happens. Student task: - Write what appears before input. - Write what value is stored or saved. - Write what output or file content should appear. - Compare with the code model. Output: Submit predicted behavior in three steps.

Activity 3: Complete The Missing Line Type: hands-on coding Time: 12 minutes Instruction: Complete one missing Java I/O line. Student task: - Add the needed import. - Add a prompt. - Add an input/read/write line. - Add a clear output line. Code / model:

```
// Complete the missing lines for this lesson focus.
// import goes here
public class PracticeIO {
    public static void main(String[] args) {
        // prompt, read/write, and output go here
    }
}
```

Output: Submit completed code or completed missing lines.

Activity 4: Test Evidence Type: individual Time: 10 minutes Instruction: Gather evidence that the program behavior is correct. Student task: - Run or trace with sample data. - Record input used. - Record console output. - If a file is involved, record file name and expected contents. Output: Submit input, output, and file evidence notes.

Activity 5: Exit Ticket Type: exit ticket Time: 5 minutes Instruction: Review Java I/O habits. Student task: - Why do prompts matter? - Why should output be labeled? - Why do file programs need testing? Output: Submit three answers.

Now We Apply

Now we apply it through these tasks: 1. Create a student introduction program. 2. Create a receipt or grade report output. 3. Save and/or read student information from a text file.

Challenge Task

Create a Student Information File Java program that reads user input, displays a clean summary, saves the information to a text file, and includes a written explanation.

How Our Work Will Be Checked

Criteria - 20 points total - Correctness (5): Output or answer follows the required concept and instructions. - Completeness (4): All required parts are present. - Logic and sequence (4): Steps, code, diagram, or pseudocode follow a clear order. - Explanation (3): Student can explain what the work does and why it is correct. - Neatness/readability (2): Work is easy to read, label, and check. - Effort and revision (2): Student improves the work after feedback.

Let's Reflect

Let's reflect: - What concept from this lesson is clearest to you now? - What mistake did you notice or correct during practice? - Which activity helped you understand the lesson best? - How can this skill help you design or write a Java program? - What do you still need to practice before the next lesson?

How We Show Learning

Evidence we will use to check learning: - Program reads required input. - Program displays clean output. - Program saves or reads a text file. - Student provides evidence and explanation. - Completed guided-practice work with visible corrections or annotations. - A short explanation connecting the output to the lesson target.

Student Activities

Input Practice. Output Formatting Practice. File I/O Concept Check. Trace File Writing Steps. Read Student Info from a File. Peer Test and Feedback. Create a student introduction program. Create a receipt or grade report output. Save and/or read student information from a text file.

Resources / References

JDK, IDE, projector or screenshots, printed worksheets, notebook, sample code snippets, and teacher-created answer guides.

LESSON 25

Lesson 25: Mini Project - Student Information File Program

What We'll Learn

By the end of this lesson, we can: - Use Java I/O concepts appropriately. - Create readable program output. - Test and explain input/output or file behavior.

Words We'll Use

- Scanner - Java class for input.
- Prompt - message asking for input.
- Output formatting - arranging results clearly.
- FileWriter - class for writing text.
- File - stored data on a computer.

Before We Start

Why is a program more useful if it can ask, display, save, or read information?

Let's Understand

Java file input and output flow guide Let's work through Lesson 25: Mini Project - Student Information File Program together. Our focus is reading input, displaying output, and saving or reading simple text files. We will move from meaning, to examples, to practice, then to a task we can explain. What this lesson means: - Input lets a Java program receive information instead of using fixed values only. - Output lets a Java program communicate clear results to the user. - Scanner can read values typed by the user or lines from a file. - FileWriter can save text into a file so the data remains after the program runs. - File I/O needs careful testing because file names, file locations, and errors matter. - Clean prompts and labeled output make our programs easier to use and check. How we study it step by step: - First, we connect the lesson to a familiar situation: Why is a program more useful if it can ask, display, save, or read information? - Next, we name the important parts so everyone uses the same vocabulary. - Then, we study Scanner and ask: what does it do, where do we see it, and how can we check it? - Then, we study Output formatting and ask: what does it do, where do we see it, and how can we check it? - Then, we study File writing and ask: what does it do, where do we see it, and how can we check it? - Then, we study File reading and ask: what does it do, where do we see it, and how can we check it? - After that, we compare a correct example with a common wrong version. - We trace the example slowly before running, drawing, or submitting anything. - Finally, we explain the result in our own words so the activity is not just copying. Rules and patterns we should remember: - Rule: Import the class before using Scanner, FileWriter, or File. Example: `import java.util.Scanner;` is needed before creating a Scanner for keyboard input. Check: Which import do we need for Scanner? - Rule: Show a clear prompt before reading user input. Example: `System.out.print("Enter your name: ");` tells the user what to type. Check: What should the user enter? - Rule: Store input in a variable with a suitable data type. Example: `String name = input.nextLine();` stores text, while `int age = input.nextInt();` stores a whole number. Check: Which method fits a name? - Rule: Label output so the user understands what the value means. Example: `System.out.println("Average: " + average);` is clearer than printing only `average`. Check: What label helps us read the result? - Rule: Close file writers or readers when we are done. Example: `writer.close();` finishes writing and releases the file. Check: What line tells Java we are done with the file? - Rule: Use simple error handling when a file might not exist or writing might fail. Example: `catch (IOException error)` lets us display a helpful message instead of crashing silently. Check: What message should the user see if saving fails? Common mistakes we will watch for: - Forgetting the import statement. - Asking for input without a clear prompt. - Reading the wrong data type. - Saving a file but not knowing where it was created. - Displaying unlabeled output that is hard to check. Mini-check before practice: - What prompt will the user see? - What variable stores the input? - What output should appear? - If a file is used, where is it saved or read from? When we move to practice, our goal is not only to get an answer. Our goal is to explain the thinking: what information we used, what step happened next, why the result is correct, and what we changed after testing.

Let's Practice

Let's practice with these activities:

Activity 1: I/O Code Walkthrough Type: individual Time: 10 minutes Instruction: Read the model and label the important input/output or file parts. Student task: - Plan input fields. - Plan output labels. - Plan file name. - Plan test evidence. Code / model:

```
Scanner input = new Scanner(System.in);
System.out.print("Enter name: ");
String name = input.nextLine();
// Save name to student.txt, then display a clean summary.
```

Output: Submit labeled code parts and one explanation.

Activity 2: Predict The Program Behavior Type: pair Time: 8 minutes Instruction: Predict what the user sees or what file action happens. Student task: - Write what appears before input. - Write what value is stored or saved. - Write what output or file content should appear. - Compare with the code model. Output: Submit predicted behavior in three steps.

Activity 3: Complete The Missing Line Type: hands-on coding Time: 12 minutes Instruction: Complete one missing Java I/O line. Student task: - Add the needed import. - Add a prompt. - Add an input/read/write line. - Add a clear output line. Code / model:

```
// Complete the missing lines for this lesson focus.
// import goes here
public class PracticeIO {
    public static void main(String[] args) {
        // prompt, read/write, and output go here
    }
}
```

Output: Submit completed code or completed missing lines.

Activity 4: Test Evidence Type: individual Time: 10 minutes Instruction: Gather evidence that the program behavior is correct. Student task: - Run or trace with sample data. - Record input used. - Record console output. - If a file is involved, record file name and expected contents. Output: Submit input, output, and file evidence notes.

Activity 5: Exit Ticket Type: exit ticket Time: 5 minutes Instruction: Review Java I/O habits. Student task: - Why do prompts matter? - Why should output be labeled? - Why do file programs need testing? Output: Submit three answers.

Now We Apply

Now we apply it through these tasks: 1. Create a student introduction program. 2. Create a receipt or grade report output. 3. Save and/or read student information from a text file.

Challenge Task

Create a Student Information File Java program that reads user input, displays a clean summary, saves the information to a text file, and includes a written explanation.

How Our Work Will Be Checked

Criteria - 20 points total - Correctness (5): Output or answer follows the required concept and instructions. - Completeness (4): All required parts are present. - Logic and sequence (4): Steps, code, diagram, or pseudocode follow a clear order. - Explanation (3): Student can explain what the work does and why it is correct. - Neatness/readability (2): Work is easy to read, label, and check. - Effort and revision (2): Student improves the work after feedback.

Let's Reflect

Let's reflect: - What concept from this lesson is clearest to you now? - What mistake did you notice or correct during practice? - Which activity helped you understand the lesson best? - How can this skill help you design or write a Java program? - What do you still need to practice before the next lesson?

How We Show Learning

Evidence we will use to check learning: - Program reads required input. - Program displays clean output. - Program saves or reads a text file. - Student provides evidence and explanation. - Completed guided-practice work with visible corrections or annotations. - A short explanation connecting the output to the lesson target.

Student Activities

Input Practice. Output Formatting Practice. File I/O Concept Check. Trace File Writing Steps. Read Student Info from a File. Peer Test and Feedback. Create a student introduction program. Create a receipt or grade report output. Save and/or read student information from a text file.

Resources / References

JDK, IDE, projector or screenshots, printed worksheets, notebook, sample code snippets, and teacher-created answer guides.